

FUSION: Forecast-Embedded Agent Scheduling with Service Incentive Optimization over Distributed Air-Ground Edge Networks

Houyi Qi, *Graduate Student Member, IEEE*, Minghui Liwang, *Senior Member, IEEE*, Seyyedali Hosseinalipour, *Senior Member, IEEE*, Liquun Fu, *Senior Member, IEEE*, Sai Zou, *Senior Member, IEEE*, Xianbin Wang, *Fellow, IEEE*, Wei Ni, *Fellow, IEEE*, and Yiguang Hong, *Fellow, IEEE*

Abstract—This paper introduces a forecasting-driven, incentive-aware service provisioning framework for distributed air-ground integrated networks with human-machine coexistence. Agent pairs (APs), each comprising a vehicle and its carried uncrewed aerial vehicles (UAVs), are proactively dispatched to overloaded hotspots to augment the computing capacity of edge servers (ESs). This design introduces four coupled challenges: uncertain spatio-temporal workloads, coupling between vehicular mobility and UAV capacity, forecast-driven contracting risks, and heterogeneous quality-of-service (QoS) requirements of human users (HUs) and machine users (MUs). To address these challenges, we propose FUSION, a two-stage framework with offline service preparation and online task scheduling. In the offline stage, a liquid neural network forecasts multi-step ES demand, an enhanced ant colony optimization scheme constructs AP service routes, and an auction-based mechanism establishes ES-AP contracts. In the online stage, we formulate congestion-aware scheduling as an exact-potential game among service demanders (SDs) and develop a potential-guided best-response dynamics algorithm. For a fixed online state, the algorithm converges to an ε -Nash equilibrium (NE) under a positive improvement threshold and to a pure-strategy NE when the threshold is zero. Within the considered contracting model, we theoretically establish that the offline mechanism satisfies individual rationality, near-truthfulness, and weak budget balance. Experiments on synthetic data and real-world load traces show that FUSION achieves higher social welfare while maintaining interaction delay and signaling energy overheads comparable to the considered benchmarks.

Index Terms—Distributed edge computing, liquid neural network, auction, potential games, heterogeneous network users.

I. INTRODUCTION

Recent years have witnessed a new generation of intelligent applications such as real-time distributed video analytics, intelligent transportation systems, and large-scale mobile crowdsensing, flourishing across smart cities [1], [2]. Although edge servers (ESs) can provide computing capabilities for these intelligent applications at the network edge, their limited local resources make them susceptible to resource shortages

and task congestion during peak demand periods [3]–[5]. To enhance the capabilities of ESs, low-altitude wireless and edge networking enabled by uncrewed aerial vehicles (UAVs) has received significant attention [6]–[9]. Unlike static ESs, UAVs offer high mobility and flexibility, enabling rapid deployment to hotspot regions. Existing UAV-assisted edge frameworks mainly optimize post-deployment operations, such as trajectory and resource scheduling, inference offloading, data uploading, and autonomous navigation [10]–[13]. However, how UAVs are transported from deployment bases to geographically distributed hotspots before service provision remains largely unexplored. To fill this gap, we propose an *air-ground cooperative edge computing paradigm*: during periods of intensive demand, ground vehicles transport UAVs to the vicinity of overloaded ESs, with each vehicle and its carried UAVs collectively referred to as an “agent pair (AP).” UAVs are then launched from strategically selected locations to provide temporary computing capacity to the ESs. Operating this hybrid AP-ES deployment paradigm introduces four main challenges:

(A) Spatio-temporal demand uncertainty. Edge computing demands exhibit strong spatio-temporal variability, and the coexistence of human users (HUs) and machine users (MUs) further complicates the management of distributed workloads. This variability makes it difficult to determine when, where, and how UAV services should be dispatched. In such networks, static provisioning or purely reactive scheduling often leads to mismatches between supply and demand, resulting in congestion at emerging hotspots and resource underutilization in low-demand regions [15], [16]. The key challenge therefore lies in *accurately forecasting spatio-temporal demand patterns from historical requests and contextual information*, so as to enable proactive UAV deployment.

(B) Spatio-temporal coupling of vehicular mobility and UAV deployment. Supporting geo-distributed ESs requires APs to rely on vehicular mobility to reach target locations, which is constrained by road topology and traffic conditions. Moreover, the number of UAVs that can be deployed by a vehicle is tightly coupled with its visiting sequence and dwell time at each ES, since UAV dispatching, service execution, and recovery jointly consume time and onboard resources. Consequently, vehicle-UAV joint deployment becomes a *highly coupled optimization problem that must simultaneously account for travel costs, service time windows, UAV dispatching logistics, and spatially distributed service rewards*.

(C) Predictive contracting and incentive-compatible trading. In the ES-AP hybrid network, APs operated by private entities must be compensated by ES operators through *advance*

H. Qi (houyiqi@tongji.edu.cn), M. Liwang (minghuiliwang@tongji.edu.cn), and Y. Hong (yghong@iss.ac.cn) are with the Shanghai Research Institute for Intelligent Autonomous Systems, with the State Key Laboratory of Autonomous Intelligent Unmanned Systems, Frontiers Science Center for Intelligent Autonomous Systems, Ministry of Education, Shanghai Key Laboratory of Intelligent Autonomous Systems, and Department of Control Science and Engineering, Tongji University, China. S. Hosseinalipour (alipour@buffalo.edu) is with the Department of Electrical Engineering, University at Buffalo-SUNY, USA. L. Fu (liquun@xmu.edu.cn) is with the School of Informatics, Xiamen University, China. S. Zou (dr-zousai@foxmail.com) is with the College of Big Data and Information Engineering, Guizhou University, China. X. Wang (xianbin.wang@uwo.ca) is with the Department of Electrical and Computer Engineering, Western University, Canada. W. Ni (Wei.Ni@ieee.org) is with School of Engineering, Edith Cowan University, Perth, Australia.

Corresponding author: Minghui Liwang

contracts that commit service capacity and pricing for future peak-demand periods before demand realization. However, forecasting errors and dynamic supply-demand variations introduce significant risks for both parties: ESs may suffer unmet demand, while APs may incur revenue loss due to resource misallocation. The central challenge is thus to design *a reliable offline trading mechanism that is individually rational, incentive compatible, and budget feasible, while operating prior to demand realization and service execution* [17].

(D) Online network coordination under human-machine coexistence. HUs and MUs compete for shared resources provided by local ESs and leased UAVs, yet exhibit fundamentally different QoS requirements. HUs are typically delay-sensitive due to interactive applications, whereas MUs emphasize timely task completion before deadlines. The resulting challenge is to design *fair and efficient distributed resource scheduling mechanisms that jointly serve HUs and MUs while ensuring stable operation and provable convergence guarantees*.

To address these challenges, we develop a forecasting-driven agent scheduling and incentive optimization (FUSION) framework for distributed service provisioning. For challenge (A), we develop proactive spatio-temporal demand prediction via liquid neural networks (*Pro-LNN*) to transform historical service traces and contextual features into multi-step workload forecasts. For challenge (B), we design enhanced ant colony optimization for vehicular route planning (*eACO-VRP*) to construct AP service routes based on the predicted demand. For challenge (C), we introduce offline auction-based incentive-compatible contracting (*Off-AIC²*) to establish ES-AP contracts and determine the corresponding payments and rewards. For challenge (D), we formulate online task scheduling as a potential game among service demanders (SDs) and develop potential game-based best-response dynamics (*PG-BRD*). Its asynchronous payoff-improving updates lead to an ε -Nash equilibrium (NE), or a pure-strategy NE when $\varepsilon = 0$. Key contributions are:

- We develop an air-ground integrated edge network model that jointly captures vehicular-UAV APs, ESs, and heterogeneous human-machine computing demands.
- We formulate ES-AP-user service provisioning under spatio-temporal mobility constraints, stochastic workload evolution, and strategic agent behaviors. The offline ES-AP contract design leads to a combinatorial mixed-integer formulation, whereas the online SD-side interactions over heterogeneous ES/UAV resources form a finite potential game with a combinatorial action space. We develop a two-stage solution consisting of a forecast-driven offline stage and a game-theoretic online stage.
- We develop FUSION to couple proactive offline service preparation with adaptive online scheduling. For the offline stage, Pro-LNN performs multi-step spatio-temporal demand forecasting, eACO-VRP constructs executable vehicle-UAV routes, and Off-AIC² establishes forecast-driven ES-AP contracts with individual rationality, near-truthfulness, and weak budget balance under the stated assumptions. For the online stage, we formulate real-time task scheduling as an SD-side potential game under human-machine coexistence and design PG-BRD to obtain congestion-aware SD-SP assignments with convergence to an ε -NE, which reduces to

TABLE I
A SUMMARY OF RELATED STUDIES
(ST: SPATIO-TEMPORAL, PRED.: PREDICTIVE, HET.: HETEROGENEOUS)

Reference	Network Characteristics			Service Delivery			Participant	
	Static	Dynamic	ST	Offline	Online	Pred.	Het. users	Het. ESs
[10]		✓	✓		✓			
[11]		✓			✓		✓	
[12]		✓		✓				✓
[13]		✓			✓			
[14]		✓	✓		✓		✓	✓
[15], [22], [23]	✓			✓				✓
[16]	✓			✓				✓
[18]		✓	✓	✓			✓	✓
[19], [20]		✓			✓		✓	✓
[21]		✓		✓			✓	
This Work		✓	✓	✓	✓	✓	✓	✓

a pure-strategy NE when $\varepsilon = 0$.

- Evaluations on synthetic data and real-world load traces show that FUSION achieves higher social welfare while maintaining interaction delay and signaling energy overheads comparable to the considered baselines.

II. LITERATURE INVESTIGATION

A. Service Provision in UAV-Assisted Edge Networks

There exists an extensive body of literature on improving the communication/computation performance and reducing the energy footprint of UAV-assisted edge computing systems [10]–[14]. For instance, Wang *et al.* [10] designed a multi-agent deep reinforcement learning (DRL)-based resource scheduling scheme for UAV-assisted wireless edge networks; Xu *et al.* [11] studied the minimization of average inference delay in UAV-assisted computing systems via DRL-based resource allocation; Li *et al.* [12] investigated distributed/federated edge intelligence for UAV-assisted networks while optimizing energy consumption and resource utilization; and Zeng *et al.* [13] introduced an ES-assisted UAV navigation system that integrates DRL-based decision making with network-aware containerized resource allocation to enhance navigation quality. More recently, Sun *et al.* [14] developed a two-timescale framework that incorporates price incentive-based service provisioning with task offloading and UAV trajectory/resource optimization. Most existing UAV-assisted edge frameworks begin with a pre-established UAV infrastructure and focus on optimizing its subsequent operation, such as online scheduling, trajectory control, and edge service provisioning. In contrast, the preceding resource provisioning stage, covering vehicle-carried UAV deployment and advance ES-AP contracting before demand realization, remains largely overlooked. Consequently, the four interrelated challenges identified in Sec. I have not been jointly addressed.

B. Auction-Driven and Game-Theoretic Service Provisioning

Auctions have been widely used for resource provisioning from service providers to users. For instance, Ren *et al.* [18] investigated truthful auctions for dependent task scheduling over dynamic edge networks; Xu *et al.* [19] designed combinatorial auction-based deep neural network (DNN) inference for heterogeneous edge-cloud networks to maximize social

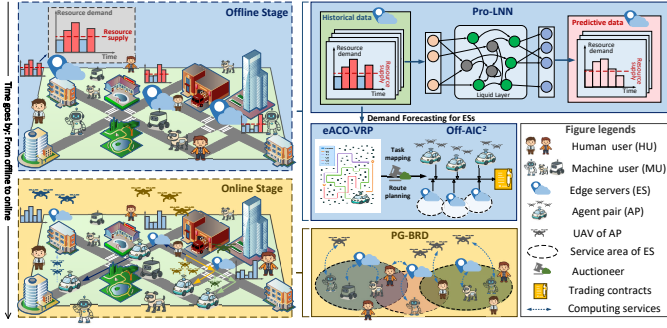


Fig. 1. Schematic of FUSION over an air-ground integrated network.

welfare; *Li et al.* [20] developed an online auction for generative content scheduling; *Lu et al.* [21] proposed auction-based distributed/federated learning to improve the distributed model training speed and energy consumption in edge networks. While the above studies provide valuable insights, they are mostly *reactive* (i.e., decisions are made after demand realization), and typically assume fixed resources and stable demand. In contrast, UAV-assisted edge computing involves dynamic resources and stringent delay/energy constraints, calling for spatio-temporally aware auctions with minimal online interactions between service providers and end users while accounting for uncertainties in both resource availability and demand. This gap is addressed by our Off-AIC² framework. Beyond auctions, potential games have also been explored as distributed optimization strategies for service provisioning [15], [16]. For example, *Ding et al.* [15] investigated task processing across user-edge-cloud networks as potential games; *Zhou et al.* [16] modeled computation offloading as a potential game with latency-energy trade-offs; *Hsieh et al.* [22] proposed a knapsack potential game for task scheduling; *Wu et al.* [23] studied task scheduling for multi-cell edge-cloud networks through potential games with NE guarantees. Despite their notable contributions, the above works typically consider a single/homogeneous user type, overlooking the fundamentally different QoS requirements of HUs and MUs, as well as their interactions with heterogeneous ESs. Our PG-BRD design considers potential-game-based scheduling in UAV-assisted networks with human-machine coexistence, spatio-temporal task windows, and heterogeneous service resources. Table I summarizes the differences between FUSION and representative existing studies.

III. OVERVIEW AND MODELING

A. Overview of Our Methodology

We consider an air-ground integrated network, wherein nodes are partitioned into *service demanders* and *service providers*.

(i) *Service demanders (SDs)*: SDs are users that generate tasks such as model training/inference¹. To capture different QoS needs under human-machine coexistence, we further classify SDs into two categories:

- **HUs**, denoted by $\mathcal{U}^H = \{u_1^H, \dots, u_{|\mathcal{U}^H|}^H\}$, are sensitive to service latency because applications such as AR/VR, interactive gaming, real-time video conferencing, and intelligent

driving are interactive and experience-driven. Even modest latency increases can reduce their perceived utility.

- **MUs**, denoted by $\mathcal{U}^M = \{u_1^M, \dots, u_{|\mathcal{U}^M|}^M\}$, correspond to autonomous devices such as robots, mapping vehicles, and sensing units. MUs primarily require their computation tasks to be completed before specified deadlines and are therefore less sensitive to instantaneous latency, provided that task feasibility is preserved (e.g., a robot's image-analysis task remains feasible if its inference result is obtained before the deadline).

(ii) *Service providers (SPs)*: We consider two types of SPs:

- **ESs**, denoted by $\mathcal{S} = \{s_1, \dots, s_{|\mathcal{S}|}\}$, are geo-distributed across the map and provide low-latency services to nearby users. Each ES operates with a limited pool of orthogonal sub-carriers and finite resource capacity, inherently constraining the number of users that can be concurrently served. During peak demand periods, hotspot ESs are particularly susceptible to resource saturation, which can trigger substantial queuing delays or even admission blocking.
- **APs**, denoted by $\mathcal{V} = \{v_1, \dots, v_{|\mathcal{V}|}\}$, are formed by vehicles carrying multiple on-board charging UAVs. Due to the limited power of UAVs, vehicles first traverse pre-planned routes to transport UAVs to designated service/ES locations. Upon arriving at their contracted ESs, UAVs are deployed to offer extra computing resources. This procedure incurs operational costs, including vehicle repositioning, UAV dispatching/launching procedures, flight energy consumption, and maintenance overhead.

Under this two-sided demand-supply structure, SDs compete for reliable computing services, while SPs collaboratively provide resources subject to their capacity and operational constraints. However, ESs face *uncertain service demand*: request peaks vary stochastically across ESs in time and space, and the HUs-to-MUs composition also changes over time. To capture these dynamics, each service window is discretized into T time slots indexed by $t \in \{0, 1, \dots, T-1\}$. We therefore design a two-stage framework that separates offline planning from online execution, as shown in Fig. 1. From the ES perspective, the offline stage (Sec. IV) precedes each trading round and uses historical records and statistical demand patterns. Since future HUs and MUs are unknown, only aggregate ES-level demand can be predicted. Accordingly, Pro-LNN forecasts future demand, which then supports AP route planning, UAV deployment, and auction-based ES-AP contracting through eACO-VRP and Off-AIC². The online stage (Sec. V) begins when actual tasks arrive and SD identities and locations become known. Given the active participants and their instantaneous requirements, real-time task-resource scheduling is performed via an exact-potential game on the SD side. Each SD updates its scheduling decision based on a potential-aligned payoff, while the resulting assignment profile implicitly determines SP-side admissions, served sets, congestion levels, and WPS-based service assignment under the corresponding coverage and capacity constraints. The induced game guarantees finite best-response convergence to an ε -NE under a positive improvement threshold, and to a pure-strategy Nash equilibrium when the threshold is zero. Thus, FUSION uses historical demand, AP mobility/resource states, and reported bids and

¹Each SD generates one computational task per trading round and may execute it locally or offload it to an accessible SP.

asks for offline route planning, capacity reservation, and ES-AP contracting, while PG-BRD adapts SD-SP assignments online based on actual task arrivals, channel conditions, SD locations, and congestion states. Appx. G further compares FUSION with small-scale full-information benchmarks to quantify the performance impact of this two-stage decomposition.

B. Modeling of SDs, SPs, and Contracts

Hereafter, we provide detailed modeling of SPs and SDs.

• *Modeling of SDs.* We model HUs and MUs as follows:

(i) **HUs:** Each HU $u_m^H \in \mathcal{U}^H$ is modeled by a 5-tuple $u_m^H = \{e_m^{H,tx}, r_m^H, d_m^H, f_m^{H,loc}, C_m^H\}$, where $e_m^{H,tx}$ is its uplink transmit power (Watts); r_m^H is its required computing workload (CPU cycles); d_m^H is its input data size (bits); $f_m^{H,loc}$ is its local CPU frequency; and C_m^H represents the set of reachable ESs within the communication coverage of u_m^H .

(ii) **MUs:** Each MU $u_n^M \in \mathcal{U}^M$ is represented by a 6-tuple $u_n^M = \{\tau_n^{ddl}, e_n^{M,tx}, r_n^M, d_n^M, f_n^{M,loc}, C_n^M\}$, where τ_n^{ddl} is its task deadline time; $e_n^{M,tx}$ is its transmit power; r_n^M and d_n^M denote its workload and data size, respectively. Also, $f_n^{M,loc}$ denotes its CPU capability, and C_n^M captures SPs that can serve u_n^M .

• *Modeling of SPs.* We model ESs and APs as follows:

(i) **ESs:** Each ES $s_i \in \mathcal{S}$ is represented by a 5-tuple: $s_i = \{t_i^{E,b}, f_i^E, e_i^E, G_i, l_i^E\}$, where $t_i^{E,b}$ denotes the time-slot at which s_i begins to request resource supply to mitigate the shortage of edge resources within its own coverage area; f_i^E is its computing capacity (e.g., CPU cycles/s); e_i^E is its local power consumption (Watts); $l_i^E = (x_i^E, y_i^E, h_i^E)$ is its 3D location. In addition, each ES s_i is presumed to have a set of orthogonal subcarriers that can serve at most G_i SDs simultaneously.

(ii) **APs:** Each AP $v_k \in \mathcal{V}$ is modeled by a 7-tuple: $v_k = \{C_k^A, f_k^A, e_k^{comp}, e_k^{move}, e_k^{fly}, G_k^A, l_k^A(t)\}$, where C_k^A denotes the maximum number of UAVs carried by the vehicle; f_k^A captures the computing capability of each mounted UAV; e_k^{move} , e_k^{comp} , and e_k^{fly} denote the power consumption of ground movement, UAV computation, and UAV flight, respectively; G_k^A denotes the maximum number of SDs that each UAV of v_k can serve through its limited orthogonal subcarriers; $l_k^A(t) = (x_k^A(t), y_k^A(t))$ is the ground location of the vehicle at time t . We further define the total online service capacity of AP v_k as $Q_k^A = C_k^A G_k^A$, which denotes the maximum number of SD service units that can be supported by all UAVs carried by AP v_k during the considered online service window.

• *Modeling of Offline Contracts.* In the offline stage, a trading contract is established between each ES s_i and AP v_k , denoted as $\mathbb{C}_{i,k}^{E \leftrightarrow A} = \{N_{i,k}^{Trad}, p_{i,k}^{ES}, r_{i,k}^{ES}\}$, where $N_{i,k}^{Trad}$ represents the quantity of traded resources, $p_{i,k}^{ES}$ indicates unit payment (per task) from s_i to contractual AP v_k , and $r_{i,k}^{ES}$ refers to the unit reward (per task) of v_k for offering services.

IV. OFFLINE STAGE: FORECASTING-DRIVEN CONTRACTING AND AP DEPLOYMENT

The offline stage converts predicted spatio-temporal demand into route-feasible and incentive-aware AP service plans. It comprises three components: (i) Pro-LNN, which forecasts workloads across distributed ESs; (ii) eACO-VRP, which constructs AP routes subject to service-window and capacity

constraints; and (iii) Off-AIC², which determines winning ES-AP contracts together with their service quantities, payments, and rewards. We let $x_{i,k}^{Off}$ denote the assignment/matching between ESs and APs, where $x_{i,k}^{Off} = 1$ indicates that ES s_i successfully secures the service of AP v_k and thereby signs an offline contract, and $x_{i,k}^{Off} = 0$ otherwise. For notational simplicity, we define $\mathbf{X}^{Off} = \{x_{i,k}^{Off} \mid s_i \in \mathcal{S}, v_k \in \mathcal{V}\}$ as the assignment profile.

A. Utilities of ESs, APs, and the Auctioneer

1) *Utility of ESs and APs:* In the offline stage, as delivering edge services to users and leasing computing capacity can generate both costs and revenues for ESs, we model the utility of each ES s_i as its net profit as follows:

$$U_i^{ES,Off} = \sum_{v_k \in \mathcal{V}} x_{i,k}^{Off} N_{i,k}^{Trad} (\bar{p}_i - p_{i,k}^{ES}), \quad (1)$$

where $\bar{p}_i$ denotes the historical average payment earned by s_i for providing edge services to users.

The utility of each AP is modeled as the monetary reward obtained from serving contracted ESs minus its mobility, flight, computation, and maintenance costs. For AP v_k serving ES s_i , the vehicle moving cost and UAV flight cost are $c_{i,k}^{move} = \tau_{i,k}^{move} e_k^{move}$ and $c_{i,k}^{fly} = \tau_{i,k}^{fly} e_k^{fly}$, respectively. The unit computation cost of processing one task is $c_{i,k}^{comp} = \frac{\bar{r} e_k^{comp}}{f_k^A}$, where $\bar{r}$ is the historical average computational workload per task. Therefore, the offline utility of AP v_k is

$$U_k^{A,Off} = \sum_{s_i \in \mathcal{S}} x_{i,k}^{Off} [N_{i,k}^{Trad} r_{i,k}^{ES} - \omega_1 (c_{i,k}^{move} + c_{i,k}^{fly} + N_{i,k}^{Trad} c_{i,k}^{comp})] - \omega_2 c_k^{hard} \mathbb{1}_{\{\sum_{s_i \in \mathcal{S}} x_{i,k}^{Off} > 0\}}, \quad (2)$$

where $\mathbb{1}_{\{\cdot\}}$ denotes the binary indicator function, ω_1 converts energy consumption into monetary cost and $\omega_2 c_k^{hard}$ captures the maintenance/depreciation cost incurred when AP v_k participates in service provisioning.

2) *Utility of the Auctioneer:* An auctioneer (e.g., a trusted third party) collects bids and asks, determines winning ES-AP associations, and establishes the corresponding trading contracts. We define the utility of the auctioneer as the difference between the payments collected from ESs and the rewards paid to APs:

$$U^{Auc,Off} = \sum_{s_i \in \mathcal{S}} \sum_{v_k \in \mathcal{V}} x_{i,k}^{Off} N_{i,k}^{Trad} (p_{i,k}^{ES} - r_{i,k}^{ES}). \quad (3)$$

B. Problem of Interest in Offline Stage of FUSION

The objective of the offline stage is to maximize the social welfare (SW), defined as the sum of the utilities of participating ESs, APs, and the auctioneer:

$$U^{SW} = \sum_{s_i \in \mathcal{S}} U_i^{ES,Off} + \sum_{v_k \in \mathcal{V}} U_k^{A,Off} + U^{Auc,Off}. \quad (4)$$

We formulate the joint selection of winning ES-AP pairs and their contracts as an offline optimization problem $\mathcal{F}^{(1)}$:

$$\mathcal{F}^{(1)} : \max_{\mathbf{X}^{Off}, \{\mathbb{C}_{i,k}^{E \leftrightarrow A}\}} U^{SW} \quad (5)$$

$$\text{s.t.} \quad \sum_{v_k \in \mathcal{V}} x_{i,k}^{Off} \leq 1, \quad \forall s_i \in \mathcal{S} \quad (5a)$$

$$x_{i,k}^{\text{Off}} \in \{0, 1\}, \quad \forall s_i \in \mathcal{S}, \forall v_k \in \mathcal{V} \quad (5b)$$

$$\sum_{s_i \in \mathcal{S}} x_{i,k}^{\text{Off}} N_{i,k}^{\text{Trad}} \leq Q_k^{\text{A}}, \quad \forall v_k \in \mathcal{V}. \quad (5c)$$

where (5a) ensures that each ES is matched with at most one AP, (5b) enforces the binary nature of $x_{i,k}^{\text{Off}}$, and (5c) guarantees that the promised resources in signed contracts do not exceed the supply of each AP v_k . Problem $\mathcal{F}^{(1)}$ represents a combinatorial mixed-integer optimization problem [24], [25] involving binary ES–AP assignment variables, continuous contract variables, and route-feasibility constraints. The problem becomes computationally challenging because ES–AP matching, service quantities, prices, and AP routes are tightly coupled. Because ES–AP contracts are established before actual task arrivals, the offline stage requires an estimate of future resource demand. We use Pro-LNN to forecast ES-side spatio-temporal demand, eACO-VRP to construct AP service routes from these forecasts, and Off-AIC² to determine the final ES–AP associations and contracts.

C. Forecasting, Agent Deployment, and Contracting

We next present the offline stage, which couples demand forecasting (Pro-LNN), AP deployment (eACO-VRP), and long-term contracting (Off-AIC²).

- At the beginning of each offline trading round, each ES runs Pro-LNN to predict its service demand over future rounds and submits the resulting demand profile as bid information. Each AP reports its available service capacity and ask price based on its computing capability, energy consumption, and service duration.
- Given the bids, asks, predicted ES demands, and AP resource states, the auction platform first invokes eACO-VRP to construct route-feasible AP service plans, specifying the ESs each AP can visit and their visiting order. Off-AIC² then determines the winning ES–AP contracts and applies its critical-price-based rules to compute the corresponding payments and rewards.
- The offline outcome consists of the ES–AP matching relations, payment vectors, and pre-planned service routes for each AP, which are passed to the online stage over the subsequent time horizon.

This ordering ensures that incentive-aware contracting is performed only over physically executable ES–AP service plans, while resulting routes, contracts, and capacities support online scheduling via PG-BRD². We describe three modules: Pro-LNN-based demand forecasting, eACO-VRP-based AP deployment, and the Off-AIC² contracting mechanism.

1) *Design of Proactive Spatio-Temporal Demand Prediction via Liquid Neural Networks (Pro-LNN)*: Pro-LNN uses a liquid neural network (LNN) [26] with neuron-wise time constants to model ES-side demand across trading rounds $\nu = 1, 2, \dots$.³ The time constants provide different memory scales for capturing short- and long-term demand variations. Demand observations may also be missing or noisy. The implementation considered here uses a fixed interval $\Delta\nu$ between trading rounds, as detailed in Appx. B.1.

²A detailed discussion on the necessity and complementarity of the FUSION components is provided in Appx. C.

³Here, ν indexes the offline trading round, and each offline trading round is followed by an online service window consisting of T online execution slots.

Let $N_i^{\text{Dem},(\nu)}$ denote the task demand of ES s_i at the ν -th trading round. We form an input vector $\mathbf{z}_i^{(\nu)} = [N_i^{\text{Dem},(\nu)}, \mathbf{x}_i^{(\nu)}]$, where $\mathbf{x}_i^{(\nu)}$ stacks optional exogenous features at round ν , such as time-of-day and day-of-week indicators, ES-type and capacity descriptors, and local mobility/load statistics extracted from recent logs. Given a length- L history of past observations $\{\mathbf{z}_i^{(\nu-L+1)}, \dots, \mathbf{z}_i^{(\nu)}\}$, Pro-LNN encodes this sequence via a liquid cell with neuron-wise time constants, integrates the measurements into a hidden state $\mathbf{h}_i^{(\nu)}$, and then applies a shallow readout network to produce an H -step forecast $\{\hat{N}_i^{\text{Dem},(\nu+1)}, \dots, \hat{N}_i^{\text{Dem},(\nu+H)}\}$. Pro-LNN is trained with an H -step mean squared error (MSE) loss between the forecasts and ground-truth demands. A multi-step consistency term penalizes prediction errors during closed-loop rollouts, as detailed in Appx. B.1. These forecasts are computed offline before each trading round and then fed into the eACO-VRP module for ES recommendation and AP route pre-planning, as well as the auction module for ES–AP contract design. The update equations and training pseudocode are provided in Appx. B.1.

2) *Enhanced Ant Colony Optimization for Vehicle Route Planning (eACO-VRP)*: After Pro-LNN predicts the upcoming service demand, FUSION uses eACO-VRP to determine whether APs can reach and serve the predicted overloaded ESs. Specifically, each AP selects and orders ESs under service-window, mobility, UAV-capacity, and travel/service-cost constraints, yielding a constrained vehicle-routing problem [28], [29]. Executed before each online service window, eACO-VRP constructs route-feasible ES–AP candidates from predicted demand and AP states, which are then passed to Off-AIC² for contract, payment, and reward determination. Online variations in HU/MU arrivals, channel states, SD locations, and congestion are handled by PG-BRD⁴. Based on this design, for a given v_k , we model v_k and all ESs as vertices of a directed complete graph $\mathcal{G}_k(\mathcal{N}_k, \mathcal{E}_k)$. Here, $\mathcal{N}_k = \{n_0, n_1, \dots, n_{|\mathcal{S}|}\}$ denotes the vertex set, where n_0 represents v_k and $n_1, \dots, n_{|\mathcal{S}|}$ denote ESs; while the edge set is $\mathcal{E}_k = \{(n_r, n_s) \mid n_r, n_s \in \mathcal{N}_k, r \neq s\}$. Let the initial location of n_0 be $l_0^s \langle 0 \rangle$, and the fixed locations of $n_1, \dots, n_{|\mathcal{S}|}$ be $l_1^s, \dots, l_{|\mathcal{S}|}^s$. We use $\mathbb{K}$ to denote the ant set: each ant $\mathbb{k} \in \mathbb{K}$ simulates the trajectory of v_k , starting from $l_0^s \langle 0 \rangle$. At each step, it evaluates current time-slot t_{cur} , travel time to each candidate ES, ES service-window beginning time, remaining UAV/service capacity, the route utility upon arrival.

For a candidate transition from n_r to n_s , let $a_s = t_{\text{cur}} + \tau_{r,s}^{\text{move}}$ be the predicted arrival time and define the service-window slack as $\text{slack}_s = t_s^{\text{E},b} - a_s$. A candidate ES is feasible only if $\text{slack}_s \geq 0$ and AP v_k still has sufficient remaining UAV/service capacity. We define the distance-based heuristic desirability as $\eta_{r,s} = \frac{1}{d_{r,s} + \epsilon_d}$, where $d_{r,s}$ is the travel distance between n_r and n_s and $\epsilon_d > 0$ avoids division by zero. The state-transition rule, i.e., the probability that ant $\mathbb{k}$ moves from n_r to n_s ($r \neq s, s \neq 0$), is

$$pr_{r,s}^{\mathbb{k}} = \begin{cases} \frac{\varphi_{r,s}^{\mathcal{E}_1} \eta_{r,s}^{\mathcal{E}_2} (\text{slack}_s + \epsilon_t)^{-\epsilon_3}}{\sum_{q \in J^{\mathbb{k}}(n_r)} \varphi_{r,q}^{\mathcal{E}_1} \eta_{r,q}^{\mathcal{E}_2} (\text{slack}_q + \epsilon_t)^{-\epsilon_3}} & \text{if } n_s \in J^{\mathbb{k}}(n_r), \\ 0 & \text{otherwise,} \end{cases} \quad (6)$$

⁴Further discussions on the responsiveness, scalability, and robustness of eACO-VRP under demand drift and topology perturbations are in Appx. B.3.

Algorithm 1: Proposed eACO-VRP

```

1 Input: AP  $v_k$  (source node  $n_0$ ), ES set  $\mathcal{S}$ , directed complete graph
    $\mathcal{G}_k(\mathcal{N}_k, \mathcal{E}_k)$ , ant set  $\mathbb{K}$ , max iterations  $iter_{\max}$ 
2 Initialization:  $\varphi_{r,s} \leftarrow \varphi_0$  for all  $(n_r, n_s) \in \mathcal{E}_k$ ;  $iter \leftarrow 1$ 
3 while  $iter \leq iter_{\max}$  do
4   for each ant  $k \in \mathbb{K}$  do
5      $M^k \leftarrow \emptyset$ ;  $n_r \leftarrow n_0$ 
6     while end condition not met do
7       Compute: feasible set  $J^k(n_r)$  and transition
         probabilities  $pr_{r,s}^k$  (via (6))
8       if  $J^k(n_r) \neq \emptyset$  then
9         sample next node  $n_s$  according to  $pr_{r,s}^k$  from
            $J^k(n_r)$ 
10         $M^k \leftarrow M^k \cup \{\text{edge}(n_r, n_s)\}$ 
11         $n_r \leftarrow n_s$ 
12      else
13        break
14     $M^{\max} \leftarrow \arg \max_{M^k: k \in \mathbb{K}} \sum_{(n_r, n_s) \in M^k} \Delta U_{r,s}^A$ 
15    for each  $(n_r, n_s) \in \mathcal{E}_k$  do
16      Pheromone evaporation and deposit (7) and (8)
17     $iter \leftarrow iter + 1$ 
18 Output:  $M^{\max}$  (pre-planned route for  $v_k$ ), and the recommended ES
    tasks along  $M^{\max}$ 

```

where $\varepsilon_1, \varepsilon_2, \varepsilon_3 > 0$ are weights and $\epsilon_t > 0$ avoids division by zero. Here, $\varphi_{r,s}$ is the pheromone level on edge (n_r, n_s) , and $\eta_{r,s}$ is the distance-based heuristic desirability. The factor $(\text{slack}_s + \epsilon_t)^{-\varepsilon_3}$ prioritizes feasible ESs with tighter service windows, while infeasible ESs with negative slack are excluded from $J^k(n_r)$. In (6), $J^k(n_r)$ denotes the feasible ES-vertex set available to ant k at node n_r : an ES vertex n_s belongs to $J^k(n_r)$ if (i) $t_{\text{cur}} + \tau_{r,s}^{\text{move}} \leq t_s^{\text{E},b}$, and (ii) AP v_k still has enough remaining UAV/service capacity to serve the ES demand. Once a route is found by ants, pheromones on edges are updated as:

$$\varphi_{r,s} \leftarrow (1 - \theta) \varphi_{r,s} + \Delta \varphi_{r,s}, \quad (7)$$

where $0 < \theta < 1$ is the evaporation coefficient. The pheromone deposit is defined as

$$\Delta \varphi_{r,s} = \begin{cases} \chi \max\{0, \Delta U_{r,s}^A\}, & \text{if } (n_r, n_s) \in M^{\max}, \\ 0, & \text{otherwise,} \end{cases} \quad (8)$$

where $\chi > 0$ is the pheromone-deposit scaling factor and $\Delta U_{r,s}^A$ denotes the utility increment for v_k when moving from n_r to serve the ES at n_s . The best path among ants is selected as $M^{\max} \in \arg \max_{M^k: k \in \mathbb{K}} \sum_{(n_a, n_b) \in M^k} \Delta U_{a,b}^A$.

For brevity, the pseudo-code of eACO-VRP is provided in Alg. 1, while its four main steps are summarized below:

Step 1. Initialization (lines 1-2, Alg. 1): Using the AP/ES information (e.g., geographic locations, the time-slot when ES s_i begins to request extra resources, AP v_k payments and service costs, and the travel/service time and energy), we construct, for each AP v_k (with source node n_0), a graph $\mathcal{G}_k(\mathcal{N}_k, \mathcal{E}_k)$ as discussed above.

Step 2. Path exploration (lines 4-13, Alg. 1): At each iteration, an ant k located at its current vertex n_r jointly evaluates the current time-slot, the travel time to each vertex, the service time at that vertex, and the time-slot when ES s_i begins to request extra resources. This screening step yields the feasible candidate set $J^k(n_r)$ and the associated transition probabilities

$pr_{r,s}^k$ as discussed above. The ant then probabilistically selects its next vertex using (6).

Step 3. Pheromone update (lines 14-16, Alg. 1): For each constructed path M^k , we compute the cumulative utility $\sum_{(n_r, n_s) \in M^k} \Delta U_{r,s}^A$. Among all ants, the path with the largest utility is selected as $M^{\max}$, and the pheromone trails on its edges are reinforced according to (7) and (8), while pheromones on other edges evaporate.

Step 4. Output (line 18, Alg. 1): After the final iteration, eACO-VRP returns $M^{\max}$ as the pre-planned route for AP v_k , together with the recommended ES service targets along this route.

3) *Offline Auction-based Incentive-Compatible Contracting (Off-AIC²):* Building on the pre-planned AP routes obtained above, we now introduce Off-AIC², an offline double-auction mechanism that establishes incentive-aware service contracts between ESs and APs. The goal is to decide ES-AP associations, how much service capacity should be reserved along each AP route, and how payments/rewards should be set so that participants (i.e., ESs and APs) have limited incentive to misreport their private values.

In Off-AIC², on the *buyer* side, each ES s_i forms its service reservation request by aggregating the H -step demand forecast as $D_i = \sum_{h=1}^H \hat{N}_i^{\text{Dem}, (v+h)}$. Here, H denotes the forecasting horizon, and D_i is the requested quantity of AP-assisted service units submitted in the current offline auction. ES s_i has a private per-unit valuation v_i for receiving edge service and participates in Off-AIC² by submitting a per-unit bid bid_i together with its reservation request D_i . Here, bid_i represents the maximum payment that s_i is willing to make per unit of service. On the *seller* side, each AP v_k , based on its total service capacity Q_k^A and its service cost, reports a per-unit ask price ask_k and its capacity Q_k^A , where ask_k is the minimum acceptable payment per unit of service⁵. Subsequently, for a potential ES-AP pair (s_i, v_k) , we define the pairwise surplus as $U_{i,k}^{\text{Off}} = \text{bid}_i - \text{ask}_k$, where $U_{i,k}^{\text{Off}} > 0$ indicates that the ES's per-unit willingness to pay exceeds the AP's per-unit ask, and hence the pair is economically admissible before route-feasibility screening. Off-AIC² then uses the reported bids, asks, demands, and capacities to compute pairwise utilities and select a set of winning ESs and APs, determine their matching/association pattern, and compute payments and rewards. The complete pseudo-code of Off-AIC² is provided in Alg. 2⁶, while its main three steps are summarized below:

Step 1. List generation and pivotal-index search (lines 3-5, Alg. 2): Off-AIC² operates as a one-shot sealed-bid double auction: all ES buyers s_i and AP sellers v_k simultaneously submit their bids, asks, and resource quantity information to an auctioneer without observing each other's submissions. We first sort ESs into a list $\mathcal{L}^{\text{ES}}$ in descending order of bid_i , and APs into a list $\mathcal{L}^{\text{AP}}$ in ascending order of ask_k .

⁵Here, ask_k denotes AP v_k 's route-aware amortized per-unit service cost, covering its expected computation, vehicle movement, UAV flight, and maintenance/depreciation costs under the route-feasible service plan in the current offline trading round.

⁶For each active contract (s_i, v_k) with $x_{i,k}^{\text{Off}} = 1$, the contract variables are set to $p_{i,k}^{\text{ES}} = p_i^{b*}$, $r_{i,k}^{\text{ES}} = r_k^{s*}$, and $N_{i,k}^{\text{Trad}} = D_i$, unless a partial-service contract is explicitly specified. Thus, the settlement vectors from Alg. 2 directly determine the payment and reward terms in $\mathbb{C}_{i,k}^{\text{E} \rightarrow \text{A}}$.

Algorithm 2: Proposed Off-AIC²

```

1 Input: ES set  $\mathcal{S}$  with reservation requests  $\{D_i\}$  and bids  $\{\overline{\text{bid}}_i\}$ ; AP
   set  $\mathcal{V}$  with asks  $\{\text{ask}_k\}$  and total service capacities  $\{Q_k^A\}$ 
2 Initialization: Set  $\mathbf{X}^* \leftarrow \mathbf{0}$ ,  $\mathbf{p}^* \leftarrow \mathbf{0}$ ,  $\mathbf{r}^{s*} \leftarrow \mathbf{0}$ . Set residual capacities
    $R_k^A \leftarrow Q_k^A$  for all APs  $v_k$ .
3 Sort ES indices into  $\mathcal{L}^{\text{ES}}$  by  $\overline{\text{bid}}_i$  (descending), and AP indices into
    $\mathcal{L}^{\text{AP}}$  by  $\text{ask}_k$  (ascending); relabel  $(i, k)$  accordingly.
4 Find a pair of pivotal ranks  $(I^*, K^*)$  such that
    $\overline{\text{bid}}_{I^*} \geq \text{ask}_{K^*}$ ,  $\sum_{k=1}^{K^*} Q_k^A \geq \sum_{i=1}^{I^*} D_i$ , and  $I^*$  is maximized
   (breaking ties by larger capacity slack  $\sum_{k=1}^{K^*} Q_k^A - \sum_{i=1}^{I^*} D_i$ ).
5 Initialize the set of unmatched ESs:  $\mathcal{U}^{\text{free}} \leftarrow \{s_i \mid 1 \leq i \leq I^*\}$ .
6 for  $k$  in top- $K^*$  APs (according to  $\mathcal{L}^{\text{AP}}$ ) do
7   if  $R_k^A \leq 0$  or  $\mathcal{U}^{\text{free}} = \emptyset$  then
8     continue
9   Build candidate ES set
    $C_k \leftarrow \{s_i \in \mathcal{U}^{\text{free}} \mid U_{i,k}^{\text{Off}} > 0 \wedge \overline{\text{bid}}_i \geq \text{ask}_k\}$ ;
10  if  $C_k = \emptyset$  then
11    continue
12   $\mathcal{K}_k^* \leftarrow \text{eACO-VRP (Alg. 1)}$ ;
13  for  $s_i \in \mathcal{K}_k^*$  do
14    Set  $x_{i,k}^{\text{Off}} \leftarrow 1$ ;
15    Remove  $s_i$  from  $\mathcal{U}^{\text{free}}$ :  $\mathcal{U}^{\text{free}} \leftarrow \mathcal{U}^{\text{free}} \setminus \{s_i\}$ ;
16  Update residual service capacity:  $R_k^A \leftarrow R_k^A - \sum_{s_i \in \mathcal{K}_k^*} D_i$ .
17 Denote the resulting matching by  $\mathbf{X}^* = \{x_{i,k}^{\text{Off}}\}$ .
18 for each matched ES  $s_i$  with  $\sum_k x_{i,k}^{\text{Off}} = 1$  do
19   Set  $\text{high} \leftarrow \overline{\text{bid}}_i$ ,  $\text{low} \leftarrow \overline{\text{bid}}_{I^*+1}$  (or a pre-specified reserve
   lower bid if  $I^* = |\mathcal{S}|$ );
20   while  $\text{high} - \text{low} > \epsilon_{\text{bin}}$  do
21     Set  $b_{\text{eff}} \leftarrow (\text{low} + \text{high})/2$ ;
22     Rerun Step 2 on the top- $I^*$  ESs and top- $K^*$  APs, using  $b_{\text{eff}}$ 
     as ES  $s_i$ 's bid, keeping all other bids/asks unchanged, and
     reinitializing temporary residual capacities as  $R_{\ell}^A \leftarrow Q_{\ell}^A$ 
     for all APs  $v_{\ell}$ , to obtain a temporary matching  $\mathbf{X}^{\text{Temp}}$ ;
23     if  $\sum_k x_{i,k}^{\text{Temp}} = 1$  then  $\text{high} \leftarrow b_{\text{eff}}$ ;
24     else  $\text{low} \leftarrow b_{\text{eff}}$ ;
25   Set  $p_i^{b*} \leftarrow \text{high}$ .
26 for each matched AP  $v_k$  with  $\sum_i x_{i,k}^{\text{Off}} \geq 1$  do
27   Set  $\text{low} \leftarrow \text{ask}_k$ ,  $\text{high} \leftarrow \text{ask}_{K^*+1}$  (or a pre-specified reserve
   upper ask if  $K^* = |\mathcal{V}|$ );
28   while  $\text{high} - \text{low} > \epsilon_{\text{bin}}$  do
29     Set  $\text{ask}_{\text{eff}} \leftarrow (\text{low} + \text{high})/2$ ;
30     Rerun Step 2 on the top- $I^*$  ESs and top- $K^*$  APs, using  $\text{ask}_{\text{eff}}$ 
     as AP  $v_k$ 's ask, keeping all other bids/asks unchanged, and
     reinitializing temporary residual capacities as  $R_{\ell}^A \leftarrow Q_{\ell}^A$ 
     for all APs  $v_{\ell}$ , to obtain a temporary matching  $\mathbf{X}^{\text{Temp}}$ ;
31     if  $\sum_i x_{i,k}^{\text{Temp}} \geq 1$  then  $\text{low} \leftarrow \text{ask}_{\text{eff}}$ ;
32     else  $\text{high} \leftarrow \text{ask}_{\text{eff}}$ ;
33   Set  $r_k^{s*} \leftarrow \text{low}$ .
34 Output: Matching  $\mathbf{X}^*$ , ES payments  $\mathbf{p}^{b*}$ , AP rewards  $\mathbf{r}^{s*}$ .

```

Without loss of generality, we relabel indices so that these sorted orders are directly reflected by i and k . The algorithm then searches for a pair of *pivotal ranks* (I^*, K^*) such that $\overline{\text{bid}}_{I^*} \geq \text{ask}_{K^*}$, $\sum_{k=1}^{K^*} Q_k^A \geq \sum_{i=1}^{I^*} D_i$.

In words, (I^*, K^*) is the largest prefix of high-bidding ESs and low-asking APs whose aggregate demand and supply can be satisfied at a non-loss-making price level.

Step 2. ES-AP matching and route planning via eACO-VRP (lines 6-16, Alg. 2): Given the pivotal ranks (I^*, K^*) , the top- I^* ESs and top- K^* APs enter the matching stage. We initialize the unmatched ES set as $\mathcal{U}^{\text{free}} = \{s_i \mid 1 \leq i \leq I^*\}$ and process APs in the order of $\mathcal{L}^{\text{AP}}$. For each AP v_k with residual capacity $R_k^A > 0$ and $\mathcal{U}^{\text{free}} \neq \emptyset$, we construct $C_k = \{s_i \in \mathcal{U}^{\text{free}} \mid U_{i,k}^{\text{Off}} >$

$0, \overline{\text{bid}}_i \geq \text{ask}_k\}$. The condition $U_{i,k}^{\text{Off}} > 0$ identifies pairs with positive reported per-unit surplus; route and capacity feasibility are evaluated by eACO-VRP.

For AP v_k , eACO-VRP searches candidate routes using ES demand, travel time, energy cost, and expected service payments. It selects an ES set $\mathcal{K}_k^* \subseteq C_k$ and a visiting order subject to the residual capacity R_k^A and service-window constraints. For each selected ES s_i , we set $x_{i,k}^{\text{Off}} = 1$ and remove s_i from $\mathcal{U}^{\text{free}}$, so that each ES is matched with at most one AP. Combining the per-AP decisions yields $\mathbf{X}^*$. Payments and rewards are determined in Step 3, and individual rationality is analyzed under the settlement assumptions in Appx. B.2.

Step 3. Pricing and contract finalization via binary search (lines 18-33, Alg. 2): Once $\mathbf{X}^*$ is determined, Off-AIC² uses binary search to approximate critical-value payments and rewards. For each participant, the search varies its report while holding the other participants' reports fixed and re-executing the matching step. In particular, for each matched ES buyer s_i (with $\sum_k x_{i,k}^{\text{Off}} = 1$), we treat $\overline{\text{bid}}_i$ as an *effective bid parameter* and search over the interval $[\overline{\text{bid}}_{I^*+1}, \overline{\text{bid}}_i]$ to find the smallest bid p_i^{b*} such that, if s_i had declared this lower bid (while all other bids/asks remained unchanged) and Step 2 was re-executed, s_i would still be selected as a winner. If $I^* = |\mathcal{S}|$, $\overline{\text{bid}}_{I^*+1}$ is replaced by a pre-specified reserve lower bid. The resulting p_i^{b*} approximates the ES's critical payment within the specified search interval. Similarly, for each matched AP seller v_k (with $\sum_i x_{i,k}^{\text{Off}} \geq 1$), we vary its effective ask within $[\text{ask}_k, \text{ask}_{K^*+1}]$ (or a pre-specified reserve upper ask if $K^* = |\mathcal{V}|$) and re-run Step 2 to determine the largest reward level r_k^{s*} such that v_k remains in the winning set. The resulting r_k^{s*} approximates the AP's critical reward within the specified search interval.

The settlement vectors $(\mathbf{p}^{b*}, \mathbf{r}^{s*})$ determine the payment and reward terms of the offline ES-AP contracts, subject to the non-deficit settlement band specified in Appx. B.2. Appx. B.2 analyzes individual rationality, near-truthfulness, and weak budget balance under the stated settlement and bounded-approximation-error assumptions.⁷ Sec. VI-C3 evaluates participant utilities under truthful and deviating reports.

V. ONLINE STAGE: POTENTIAL GAMES FOR SERVICE SCHEDULING UNDER HUMAN-MACHINE COEXISTENCE

Unlike the offline stage, the online stage handles *real-time* service requests from HUs and MUs, whose number and composition may deviate from offline forecasts. Thus, the offline outcome can only reserve feasible UAVs and service capacities for heavily loaded ESs, but cannot determine the real-time SD-SP assignment, service timing, or service rate. Therefore, we build on the offline contracts, AP routes, and capacity budgets generated before the online service window starts, and design an online SD-side assignment mechanism. This dynamically refines SD-SP associations, while the corresponding SP-side served sets, congestion levels, and resource shares are induced by the SD assignment profile, capacity constraints, and weighted processor sharing (WPS)-based sharing rule. In particular, we formulate the online PG-BRD as an SD-side assignment game.

⁷The route-aware allocation is not guaranteed to be monotone. The near-truthfulness analysis therefore relies on bounded allocation and pricing errors rather than strict dominant-strategy truthfulness.

The strategic decision-makers are the online SDs, including HUs and MUs, collected as $\mathcal{U}^{\text{On}} = \{u_1^{\text{On}}, \dots, u_j^{\text{On}}, \dots, u_{|\mathcal{U}^{\text{On}}|}^{\text{On}}\}$. The SPs, collected as $\mathcal{S}^{\text{On}} = \{s_1^{\text{On}}, \dots, s_i^{\text{On}}, \dots, s_{|\mathcal{S}^{\text{On}}|}^{\text{On}}\}$ ⁸, act as service-side entities that provide computing resources subject to coverage, capacity, and WPS-based sharing rules, but they are not independent best-response players in PG-BRD.

Each SD u_j^{On} chooses either local execution or one accessible SP for service, i.e., $\pi_j \in \mathcal{A}_j^{\text{On}} = \{0\} \cup \mathcal{C}_j$, where $\pi_j = 0$ denotes local execution, and $\mathcal{C}_j$ denotes the set of SPs accessible to u_j^{On} in the current online slot. Therefore, the online decision variable optimized by PG-BRD is the SD assignment profile $\pi = (\pi_j)_{u_j^{\text{On}} \in \mathcal{U}^{\text{On}}}$. For implementation, the scalar assignment π_j can equivalently be represented by a binary indicator vector

$$\mathbf{X}_j^{\text{On}} = (x_{j,0}^{\text{On}}, \dots, x_{j,i}^{\text{On}}, \dots, x_{j,|\mathcal{S}^{\text{On}}|}^{\text{On}}) \in \{0, 1\}^{1+|\mathcal{S}^{\text{On}}|}, \quad (9)$$

where $x_{j,0}^{\text{On}} = 1$ indicates local execution and $x_{j,i}^{\text{On}} = 1$ indicates offloading to SP s_i^{On} . Once π is given, each SP's served SD set, load level, capacity feasibility, and WPS-based resource share are determined by the model. Thus, SP-side service acceptance and resource sharing are induced quantities rather than additional online strategic actions. We next formalize this joint effect through a delay and congestion model, which will serve as the basis for defining the utilities of SDs and SPs.

A. Utility of SDs and SPs

1) *Delay and Congestion Model:* We consider the SP set $\mathcal{S}^{\text{On}}$ and the SD set $\mathcal{U}^{\text{On}} = \mathcal{U}^{\text{H}} \cup \mathcal{U}^{\text{M}}$. Each SP s_i^{On} has computing capacity $f_i^{\text{On}} > 0$ (CPU cycles/s). The SDs assigned to this SP form the set $\mathcal{U}_i^{\text{On}} = \{u_v^{\text{On}} \in \mathcal{U}^{\text{On}} : x_{v,i}^{\text{On}} = 1\}$. We adopt weighted processor sharing (WPS) [16] to allocate computing capacity among these SDs. Each associated SD receives a positive weight $\gamma_{j,i}$ determined by its service class, task urgency, fairness quota, or contractual priority. These weights are fixed during each PG-BRD run. The effective computing rate allocated to u_j^{On} is $\hat{f}_{j,i} = f_i^{\text{On}} \frac{\gamma_{j,i}}{\sum_{u_v^{\text{On}} \in \mathcal{U}_i^{\text{On}}} \gamma_{v,i}}$, and the corresponding edge-side computing delay can be obtained:

$$t_{j,i}^{\text{comp}} = \frac{r_j}{\hat{f}_{j,i}} = \frac{r_j}{\gamma_{j,i}} \times \frac{\sum_{u_v^{\text{On}} \in \mathcal{U}_i^{\text{On}}} \gamma_{v,i}}{f_i^{\text{On}}}, \quad (10)$$

where r_j denotes the computing workload (e.g., CPU cycles needed) of u_j^{On} . This WPS model compactly captures how the delay of each SD grows with the aggregate load at the SP. We further summarize the impact of associated SDs on s_i^{On} through an *effective congestion load* defined as $y_i = \sum_{u_v^{\text{On}} \in \mathcal{U}_i^{\text{On}}} \gamma_{v,i}$. With this notation, (10) can be written in a factorized form as:

$$t_{j,i}^{\text{comp}} = \theta_{j,i} \tau_i(y_i), \quad \theta_{j,i} = \frac{r_j}{\gamma_{j,i}}, \quad \tau_i(y) = \frac{y}{f_i^{\text{On}}}, \quad (11)$$

where $\theta_{j,i}$ is a user-specific load factor, whereas $\tau_i(y)$ captures the server-side congestion effect as a function of the aggregate

load. This separable structure will be instrumental for constructing the potential-aligned payoff and the exact game potential in the subsequent analysis.

While (11) characterizes the congestion-induced computing delay, an offloading SD also experiences wireless transmission latency in the uplink: when u_j^{On} offloads to s_i^{On} , the uplink delay is $t_{j,i}^{\text{tx}} = \frac{d_j}{W \log_2(1 + e^{\text{tx}} \Gamma_{j,i})}$, where d_j is the input data size, W is the bandwidth, $e^{\text{tx}} \Gamma_{j,i}$ is the signal-to-noise ratio (SNR) between u_j^{On} and s_i^{On} . Consequently, the total edge-side latency experienced by u_j^{On} when uploading its workload to s_i^{On} is:

$$T_{j,i}^{\text{edge}} = t_{j,i}^{\text{comp}} + t_{j,i}^{\text{tx}}. \quad (12)$$

2) *Utility of HUs:* Building on the above latency and energy models, we now characterize the utility of an HU when deciding whether to execute a task locally or offload it to an SP. Consider an HU $u_j^{\text{On}} \in \mathcal{U}^{\text{H}}$ with local CPU frequency f_j^{loc} and local computational power e_j^{loc} . The local execution time and energy consumption of its task are given by $t_j^{\text{loc}} = r_j / f_j^{\text{loc}}$ and $E_j^{\text{loc}} = r_j e_j^{\text{loc}} / f_j^{\text{loc}}$, respectively. When the HU u_j^{On} offloads its task to s_i^{On} , it experiences latency $t_{j,i}^{\text{comp}} + t_{j,i}^{\text{tx}}$ and uplink energy $E_{j,i}^{\text{tx}}$. Compared with local task execution, this leads to the latency saving of $t_{j,i}^{\text{save}} = t_j^{\text{loc}} - (t_{j,i}^{\text{comp}} + t_{j,i}^{\text{tx}})$ and energy saving of $c_{j,i}^{\text{save}} = E_j^{\text{loc}} - E_{j,i}^{\text{tx}}$ ($E_{j,i}^{\text{tx}}$ denotes the uplink transmission energy).

To map these latency and energy savings into a scalar benefit, we introduce non-negative valuation coefficients $\mathbb{V}_t^{\text{H}}$ and $\mathbb{V}_e^{\text{H}}$ to capture HU sensitivities to latency and energy, respectively. The HU's perceived valuation of being served by s_i^{On} is then modeled via $v_{j,i}^{\text{H}} = \mathbb{V}_t^{\text{H}} t_{j,i}^{\text{save}} + \mathbb{V}_e^{\text{H}} c_{j,i}^{\text{save}}$. Given this valuation, the HU's net utility must also account for the service payment: if s_i^{On} charges a price $p_{j,i}$ for serving u_j^{On} , its net utility is

$$U_{j,i}^{\text{H}} = v_{j,i}^{\text{H}} - p_{j,i}, \quad U_{j,0}^{\text{H}} = 0, \quad (13)$$

where the value of $U_{j,0}^{\text{H}}$ corresponds to local task execution.

3) *Utility of MUs:* While HUs mainly care about the joint latency–energy tradeoff, MUs (e.g., robots or sensing devices) are typically driven by task deadlines and reliability requirements. We therefore adopt a slightly different utility structure for MUs. Specifically, an MU $u_j^{\text{On}} \in \mathcal{U}^{\text{M}}$ is associated with a latency deadline τ_j^{ddl} , while its total edge-side latency at s_i^{On} is $T_{j,i}^{\text{edge}}$ in (12). To capture the fact that the MU primarily values meeting this deadline, we adopt a *soft-deadline* valuation

$$G_{j,i}^{\text{M}} = v_j [\tau_j^{\text{ddl}} - T_{j,i}^{\text{edge}}]_+ \times (\tau_j^{\text{ddl}})^{-1}, \quad (14)$$

where $[x]_+ = \max\{x, 0\}$ and $v_j > 0$ denotes the MU's task reward scale. This valuation is maximal when the edge-side latency is zero, decreases linearly as the latency consumes the available deadline slack, remains positive before the deadline, and becomes zero once the deadline is violated. In particular, “soft-deadline” means that the reward is continuous and piecewise linear with respect to latency, rather than being a discontinuous hard-threshold reward.

In addition to timeliness, an MU also benefits from saving local computing energy by task offloading. Similar to HUs, the MU's energy saving when utilizing edge services is $c_{j,i}^{\text{M,save}} = E_j^{\text{loc}} - E_{j,i}^{\text{tx}}$, where E_j^{loc} is the local execution energy and $E_{j,i}^{\text{tx}}$

⁸The online SP set $\mathcal{S}^{\text{On}}$ contains both native ES service units and UAV-assisted service units made available by the offline ES–AP contracts. Specifically, when an AP is contracted to support an ES during the current service window, its deployed UAV resources are treated as additional online SP resources with computing capacity, subcarrier capacity, and coverage determined by the corresponding offline route and contract.

is the uplink transmission energy. Let $\mathbb{V}_e^M$ denote the MU's valuation coefficient on energy saving. Combining deadline satisfaction, energy saving, and the service payment $p_{j,i}$, the MU's utility when offloading its task to $s_{j,i}^{\text{On}}$ is given by

$$U_{j,i}^M = G_{j,i}^M + \mathbb{V}_e^M c_{j,i}^{M,\text{save}} - p_{j,i}, \quad U_{j,0}^M = 0, \quad (15)$$

where $U_{j,0}^M$ corresponds to local task execution.

Note that both HU and MU utilities in (13) and (15) inherently share the same congestion component $t_{j,i}^{\text{comp}} = \theta_{j,i} \tau_i(y_i)$ in (11). This shared structure will be crucial for aligning SD-side updates with the exact game potential in our subsequent analysis (Sec. V-B).

4) *Cost of SPs*: Having specified how HUs and MUs (i.e., SDs) evaluate edge services, we now focus on the SPs' perspective. As more SDs offload their tasks to an SP, the SP consumes more energy and experiences higher operating costs, which must be reflected in its payoff and, ultimately, in the system-level SW. Subsequently, for each SP s_i^{On} , given its aggregated computing load $\mathbb{x}_i = \sum_{u_v^{\text{On}} \in \mathcal{U}_i^{\text{On}}} r_v$, we model its computation/operation cost as follows:

$$K_i(\mathbb{x}_i) = \omega_1 e_i^{\text{comp}} \frac{\mathbb{x}_i}{f_i^{\text{On}}} + \omega_2 c_i^{\text{hard}} \mathbb{1}_{\{\mathbb{x}_i > 0\}}, \quad (16)$$

where e_i^{comp} is the average computing power consumption of s_i^{On} , and c_i^{hard} captures hardware- and maintenance-related costs ($\omega_1, \omega_2 \geq 0$ are weighting coefficients). In (16), the term proportional to $\mathbb{x}_i / f_i^{\text{On}}$ reflects that processing a larger workload leads to higher energy and operational expenditure, while the indicator term charges maintenance/depreciation only when the SP actively serves at least one SD. It is worth noting that service payments have different roles in welfare evaluation and game dynamics. When evaluating transfer-free system welfare, payments between SDs and SPs cancel out as internal transfers. However, in the SD-side best-response dynamics, posted prices affect individual action choices and must be included in the game potential. We therefore distinguish the transfer-free welfare $\mathcal{W}(\pi)$ from the game potential $\Phi^G(\pi)$ constructed in Sec. V-B.

SP resources are limited, and SD decisions are coupled through congestion and capacity constraints. We therefore formulate online scheduling as a game in which SDs update their assignments using a potential-aligned payoff. This formulation supports distributed assignment updates while accounting for their effects on shared resources.

B. Exact Potential Function

In order to analyze our online resource scheduling game and to enable simple distributed best-response dynamics with provable convergence, we construct an exact game potential $\Phi^G(\pi)$ aligned with the potential-guided payoff used by PG-BRD. This construction leverages the unified congestion structure in (11), the active-regime form of the MU soft-deadline utility, and fixed posted prices that are determined before the PG-BRD procedure for a given online scheduling instance.

For an MU u_j^{On} served by $s_{j,i}^{\text{On}}$, the soft-deadline valuation in (14) is non-increasing in the congestion load y_i . In the active regime, it decreases linearly with congestion, yielding the decomposition in (18); outside this regime, it remains zero. For

HUs, the utility in (13) can be decomposed into a congestion-independent valuation part and a congestion-dependent penalty:

$$U_{j,i}^H = \underbrace{\mathbb{V}_t^H(t_{j,i}^{\text{loc}} - t_{j,i}^{\text{tx}}) + \mathbb{V}_e^H(E_j^{\text{loc}} - E_{j,i}^{\text{tx}})}_{S_j(i) \text{ (valuation independent of } y_i)} - \underbrace{\mathbb{V}_t^H \theta_{j,i} \tau_i(y_i)}_{\text{congestion penalty}} - p_{j,i}, \quad (17)$$

where $S_j(i)$ collects all valuation terms that do not depend on the congestion load y_i , while the second term captures how queueing-induced delay degrades the HU's utility. The fixed payment $p_{j,i}$ is written separately to distinguish the posted-price term from the service valuation.

MU utilities also admit the same structural form in the active soft-deadline regime (see Appx. E.4). In this regime, the MU utility can be rewritten as:

$$U_{j,i}^M = \tilde{S}_j(i) - \kappa_j \theta_{j,i} \tau_i(y_i) - p_{j,i}, \quad (18)$$

where $\tilde{S}_j(i)$ gathers the non-congestion valuation terms (soft-deadline valuation base reward and energy saving), and $\kappa_j > 0$ reflects the MU's effective sensitivity to congestion. As in (17), the posted price is kept separate from the non-congestion valuation term.

Considering both HUs and MUs, we collect their non-congestion service valuation into $S_j(i)$, where $i = 0$ denotes local execution and $S_j(0) = 0$. For HUs, $S_j(i)$ is obtained from the latency-energy valuation in (17); for MUs, it is obtained from the soft-deadline valuation in (18) under the active soft-deadline regime. To align selfish SD updates with a global potential, PG-BRD uses a potential-aligned scheduling payoff rather than the raw transfer-inclusive utility alone. Given the actions of all other SDs, define $y_i^{-j} = \sum_{v \neq j: \pi_v = i} \gamma_{v,i}$, $\mathbb{x}_i^{-j} = \sum_{v \neq j: \pi_v = i} r_v$. For an offloading action $i \in \mathcal{S}^{\text{On}}$, define

$$\begin{aligned} \tilde{U}_{j,i}(\pi_{-j}) &= S_j(i) - \int_{y_i^{-j}}^{y_i^{-j} + \gamma_{j,i}} \tau_i(z) dz \\ &\quad - \left[K_i(\mathbb{x}_i^{-j} + r_j) - K_i(\mathbb{x}_i^{-j}) \right] - p_{j,i}, \end{aligned} \quad (19)$$

and set $\tilde{U}_{j,0} = 0$ for local execution. Here, $p_{j,i}$ is fixed during the PG-BRD procedure for the considered online state. The integral term prices the marginal congestion contribution of SD u_j^{On} , while the bracketed term prices its marginal SP-side operating cost. Therefore, a unilateral increase in $\tilde{U}_{j,i}$ is exactly aligned with the change in the game potential defined below.

To describe how individual decisions interact at the system-level, we collect all SD assignments into the *joint assignment profile*⁹ $\pi = (\pi_j)_{j \in \mathcal{U}^{\text{On}}}$, where $\pi_j \in \{0\} \cup \mathcal{S}^{\text{On}}$ denotes the choice of u_j^{On} : $\pi_j = 0$ means that the task is processed locally at u_j^{On} , while $\pi_j = i$ indicates task offloading to SP s_i^{On} . Given a profile π , we can then compute, for each SP s_i^{On} , the effective congestion load and the physical computing load as follows:

$$y_i(\pi) = \sum_{j: \pi_j = i} \gamma_{j,i}, \quad \mathbb{x}_i(\pi) = \sum_{j: \pi_j = i} r_j, \quad (20)$$

where $y_i(\pi)$ feeds into the congestion-induced delay via $t_{j,i}^{\text{comp}} = \theta_{j,i} \tau_i(y_i(\pi))$ in (11), and $\mathbb{x}_i(\pi)$ determines the SP-

⁹Given π , the binary indicators $x_{j,i}^{\text{On}}$ are induced by $x_{j,i}^{\text{On}} = \mathbb{1}_{\{\pi_j = i\}}$, $i \in \{0\} \cup \mathcal{S}^{\text{On}}$, so that the vector $\mathbf{X}_j^{\text{On}}$ and the scalar action π_j are equivalent representations of u_j^{On} 's decision.

side computation/operation cost through $K_i(\mathbb{x}_i(\pi))$ in (16). Thus, any unilateral change of π_j only modifies the loads of the involved SPs in (20), and these updated loads propagate to SD delays and SP costs through (11) and (16), respectively.

In order to analyze the resulting game and to show that simple myopic best-response updates can converge, we construct the following game potential:

$$\begin{aligned} \Phi^G(\pi) = & \sum_{j: \pi_j \neq 0} S_j(\pi_j) - \sum_{i \in \mathcal{S}^{\text{On}}} \int_0^{y_i(\pi)} \tau_i(z) dz \\ & - \sum_{i \in \mathcal{S}^{\text{On}}} K_i(\mathbb{x}_i(\pi)) - \sum_{j: \pi_j \neq 0} p_{j, \pi_j}. \end{aligned} \quad (21)$$

The first term aggregates SD-side non-congestion service valuations, the second term captures aggregate congestion, the third term captures SP-side operating costs, and the last term accounts for fixed posted prices that directly affect SD-side best responses. For system-efficiency evaluation, we also use the transfer-free welfare $\mathcal{W}(\pi) = \sum_{j: \pi_j \neq 0} S_j(\pi_j) - \sum_{i \in \mathcal{S}^{\text{On}}} \int_0^{y_i(\pi)} \tau_i(z) dz - \sum_{i \in \mathcal{S}^{\text{On}}} K_i(\mathbb{x}_i(\pi))$, which differs from $\Phi^G(\pi)$ only by excluding monetary transfers.

Proposition 1 (Exact potential of the online scheduling game). *Consider the online scheduling game in which each SD updates its action according to the potential-aligned payoff in (19). For any SD u_j^{On} and any unilateral deviation from j to j' , we have*

$$\tilde{U}_{j, j'}(\pi_{-j}) - \tilde{U}_{j, j}(\pi_{-j}) = \Phi^G(\pi_{-j}, j') - \Phi^G(\pi_{-j}, j). \quad (22)$$

Hence, $\Phi^G(\pi)$ is an exact potential function. Any unilateral payoff-improving update strictly increases $\Phi^G(\pi)$.

Since each SD has a finite feasible action set, any sequence of strict better-response updates under (19) terminates in finitely many steps at a pure-strategy Nash equilibrium when $\varepsilon = 0$, and at an ε -NE when PG-BRD uses a positive acceptance threshold ε . The proof is given in Appx. E.

C. Problem Formulation

In the online stage, we optimize the SD assignment profile π subject to coverage and subcarrier-capacity constraints. SP-side served sets, congestion levels, and WPS resource shares are induced by this profile. Since any maximizer of $\Phi^G(\pi)$ admits no profitable unilateral feasible deviation, we formulate the online scheduling problem as $\mathcal{F}^{(\text{II})}$:

$$\mathcal{F}^{(\text{II})} : \max_{\pi} \Phi^G(\pi) \quad (23)$$

$$\text{s.t. } \pi_j \in \{0\} \cup \mathcal{S}^{\text{On}}, \quad \forall u_j^{\text{On}} \in \mathcal{U}^{\text{On}}, \quad (23a)$$

$$x_{j, i}^{\text{On}} = 0, \quad \text{if } s_i^{\text{On}} \notin C_j, \quad \forall u_j^{\text{On}} \in \mathcal{U}^{\text{On}}, \quad (23b)$$

$$\sum_{u_j^{\text{On}} \in \mathcal{U}^{\text{On}}} x_{j, i}^{\text{On}} \leq G_i, \quad \forall s_i^{\text{On}} \in \mathcal{S}^{\text{On}}, \quad (23c)$$

where $\pi = (\pi_j)_{j \in \mathcal{U}^{\text{On}}}$ denotes the joint assignment profile, with $\pi_j = 0$ representing local task execution at u_j^{On} and $\pi_j = i$ indicating task offloading to SP s_i^{On} . The binary variables $x_{j, i}^{\text{On}}$ are induced by π as defined in Sec. V-B (i.e., $x_{j, i}^{\text{On}} = 1$ if $\pi_j = i$).

Constraint (23a) enforces each SD to choose only one execution option for its current task: either local processing ($\pi_j = 0$) or offloading to a single SP ($\pi_j \in \mathcal{S}^{\text{On}}$). Constraint (23b)

Algorithm 3: Proposed PG-BRD

```

1 Input: SD set  $\mathcal{U}^{\text{On}}$ , SP set  $\mathcal{S}^{\text{On}}$ , coverage sets  $\{C_j\}$ ,
   subcarrier-capacity  $\{G_i\}$ , prices  $\{p_{j, i}\}$ , WPS weights  $\{\gamma_{j, i}\}$ , max
   iterations  $T_{\text{max}}$ , threshold  $\varepsilon$ .
2 Initialization: Construct a random feasible profile  $\pi = (\pi_j)_{j \in \mathcal{U}^{\text{On}}}$ 
   that satisfies constraints (23a)–(23c); set  $t \leftarrow 0$  and
   converged  $\leftarrow$  false.
3 while not converged and  $t < T_{\text{max}}$  do
4   converged  $\leftarrow$  true;
5   Generate a random permutation  $\mathcal{P}$  of indices in  $\mathcal{U}^{\text{On}}$ ;
6   for each SD  $u_j^{\text{On}}$  in  $\mathcal{P}$  do
7     let  $a_j^{\text{cur}} \leftarrow \pi_j$ ;
8     for all  $i$ , compute the loads excluding  $u_j^{\text{On}}$ :  $y_i^{-j}$ ,  $x_i^{-j}$ , and
        $n_i^{-j}$ ;
9     build the feasible action set  $\mathcal{A}_j^{\text{feas}}$  according to (24);
10    compute current payoff  $\tilde{U}_j(a_j^{\text{cur}} | \pi_{-j})$  via (19);
11    for each  $a \in \mathcal{A}_j^{\text{feas}}$  do
12      temporarily set  $\pi_j \leftarrow a$  and evaluate  $\tilde{U}_j(a | \pi_{-j})$ ;
13    let  $a_j^{\text{best}} \in \arg \max_{a \in \mathcal{A}_j^{\text{feas}}} \tilde{U}_j(a | \pi_{-j})$ ;
14    if  $\tilde{U}_j(a_j^{\text{best}} | \pi_{-j}) - \tilde{U}_j(a_j^{\text{cur}} | \pi_{-j}) > \varepsilon$  then
15       $\pi_j \leftarrow a_j^{\text{best}}$ ;
16      converged  $\leftarrow$  false;
17    else
18       $\pi_j \leftarrow a_j^{\text{cur}}$ ;
19   $t \leftarrow t + 1$ ;
20 Output: Final feasible profile  $\pi^* = \pi$  and convergence flag
    converged.

```

captures the service-coverage feasibility: C_j denotes the set of SPs that can physically serve u_j^{On} (e.g., within communication range), and thus the association variable for an SD outside the coverage of s_i^{On} is forced to zero. Constraint (23c) limits the admission load at each SP, where G_i is the maximum number of SDs that SP s_i^{On} can concurrently serve.

Problem $\mathcal{F}^{(\text{II})}$ is an integer programming (IP) problem over a discrete action space and is, in general, NP-hard. Since this online scheduling has to be repeatedly solved at each trading round under stringent latency and computational constraints at the edge, running a centralized IP solver or heavyweight meta-heuristics to seek a global optimum is impractical. Instead, by recasting $\mathcal{F}^{(\text{II})}$ as an exact-potential game with potential $\Phi^G(\pi)$, we obtain a formulation that admits a fully distributed implementation. In particular, PG-BRD (Sec. V-D) lets each SD myopically maximize the potential-aligned payoff in (19) subject to feasibility. Every accepted update monotonically increases $\Phi^G(\pi)$; if PG-BRD terminates with converged = **true**, the resulting profile is an ε -NE, and it becomes a pure-strategy NE when $\varepsilon = 0$ (also see Appx. D and Appx. E). Thus, PG-BRD provides a feasible distributed solution to (23), with the stated equilibrium guarantee when the convergence condition is satisfied.

D. Potential-Guided Best-Response Dynamics (PG-BRD)

For brevity, we provide the complete pseudo-code of PG-BRD in Alg. 3 and summarize its four main steps below:

Step 1. Random-feasible initialization (lines 1-2, Alg 3): We first construct a feasible initial association profile $\pi^{(0)} \in \{0, 1, \dots, |\mathcal{S}^{\text{On}}|\}^{|\mathcal{U}^{\text{On}}|}$, where $\pi_j = 0$ corresponds to local task

execution at u_j^{On} and $\pi_j = i > 0$ means that u_j^{On} offloads its task to SP s_i^{On} . For each SD u_j^{On} , we randomly select an action from the set $\{0\} \cup C_j$ and only accept it if the resulting profile satisfies the coverage constraint (23b) and the subcarrier-capacity constraint (23c). This procedure ensures that the algorithm starts from a valid resource scheduling configuration with no constraint violations.

Step 2. Feasible-action enumeration under coverage and subcarrier-capacity constraints (lines 7-9, Alg. 3): Given a current profile π , we compute for each SP s_i^{On} the effective congestion load $y_i(\pi)$ and the physical computing load $x_i(\pi)$ as in (20), as well as the number of currently served SDs $n_i(\pi) = |\{j \mid \pi_j = i\}|$. When it is SD u_j^{On} 's turn to update its decision, we temporarily remove its own contribution and obtain the loads generated by all other SDs, denoted by y_i^{-j} , x_i^{-j} , and n_i^{-j} . These quantities represent the congestion level and computing load at each SP if u_j^{On} were not associated with any SP. Given these congestion and load levels generated by all other SDs, we construct the feasible action set for u_j^{On} as:

$$\mathcal{A}_j^{\text{feas}} = \left\{ a \in \{0\} \cup \mathcal{S}^{\text{On}} \mid a = 0 \text{ or } (a = i > 0, s_i^{\text{On}} \in C_j, n_i^{-j} + 1 \leq G_i) \right\}, \quad (24)$$

where $a = 0$ corresponds to local task execution, and an offloading action $a = i > 0$ is feasible only if SP s_i^{On} covers u_j^{On} and still has enough residual subcarrier-capacity to admit one more SD without violating its capacity G_i . This step explicitly removes any candidate action that would break either the coverage or the subcarrier-capacity constraints.

Step 3. Asynchronous best response under the exact potential (lines 10-18, Alg. 3): In each iteration, SDs are visited one-by-one in a random order, and at most one SD is allowed to change its decision at a time. For a given SD u_j^{On} and current profile π , we first evaluate its current potential-aligned payoff $\tilde{U}_j(\pi_j \mid \pi_{-j})$ using (19). Then, for each feasible action $a \in \mathcal{A}_j^{\text{feas}}$, we temporarily set $\pi_j = a$, recompute the congestion and delay quantities for this SD, and obtain the resulting utility $\tilde{U}_j(a \mid \pi_{-j})$. The SD then chooses a best response $a_j^{\text{best}} \in \arg \max_{a \in \mathcal{A}_j^{\text{feas}}} \tilde{U}_j(a \mid \pi_{-j})$. To avoid frequent switches caused by marginal gains, we introduce an acceptance threshold $\varepsilon > 0$ and let u_j^{On} update its decision only if $\tilde{U}_j(a_j^{\text{best}} \mid \pi_{-j}) - \tilde{U}_j(\pi_j \mid \pi_{-j}) > \varepsilon$. Otherwise, it keeps its current action π_j unchanged. Because the game admits the exact potential $\Phi^G(\pi)$, every accepted update in this step strictly increases $\Phi^G(\pi)$.

Step 4. Convergence and resulting equilibrium (lines 3-20, Alg. 3): The above two steps (feasible-action enumeration and asynchronous best responses) are repeatedly applied by going over all SDs. When a complete pass over $\mathcal{U}^{\text{On}}$ produces no accepted update, no SD can improve its potential-aligned payoff by more than ε through a unilateral action change. Therefore, the algorithm reaches an ε -NE with respect to (19); when $\varepsilon = 0$, this reduces to a pure-strategy NE. If $T_{\max}$ is reached before the convergence condition is satisfied, PG-BRD returns the final feasible profile obtained within the iteration budget; the equilibrium guarantee holds only upon convergence.

VI. EVALUATIONS

We next conduct experiments to evaluate FUSION. We consider a service provisioning region of size 1000 m $\times$ 1000 m,

where ESs are randomly placed (HUs and MUs are in turn randomly placed within the coverage areas of ESs) and APs can be dispatched to assist overloaded ESs. Availability windows of ESs are randomly generated over a normalized time horizon to capture the temporal variability of computation demand. Other key parameters are as follows [25], [28], [32]: $f_i^E \in [1, 3] \times 10^{12}$ CPU cycles/s (this is the aggregated clock speed of CPUs co-located at the ES), $f_k^A \in [1, 3] \times 10^{10}$ CPU cycles/s, $f_m^{\text{H,loc}} = f_n^{\text{M,loc}} \in [1, 2] \times 10^9$ CPU cycles/s, $d_m^{\text{H}} \in [10, 55]$ Mbit, $d_n^{\text{M}} \in [100, 550]$ Mbit, $G_i \in [6, 8]$, $G_k^A \in [1, 2]$, $C_k^A \in [8, 10]$, $W = 50$ MHz, $e_m^{\text{H,tx}} = e_n^{\text{M,tx}} \in [0.2, 0.4]$ W, $r_m^{\text{H}} = 600$ cycles/bit $\times d_m^{\text{H}}$, and $r_n^{\text{M}} = 600$ cycles/bit $\times d_n^{\text{M}}$. Channel gains and data rates are generated using the distance-dependent path-loss models following [33]. Under these models, most links fall into an SNR range of 10–25 dB, and UAV links achieve slightly higher average SNR due to their higher line-of-sight (LoS) probability. Unless otherwise specified, the plotted simulations are averaged over 100 independent Monte Carlo trials.

A. Benchmark Methods and Evaluation Metrics

We evaluate FUSION from two complementary angles: (i) prediction performance on service demand forecasting module (Sec. VI-B); and (ii) system-level performance (Sec. VI-C).

Benchmarks and Metrics for Demand Forecasting: We compare Pro-LNN against two widely used prediction models.

- **LSTM-driven demand prediction [30]:** A standard long short-term memory (LSTM) network is applied to the input sequences $\{z_t\}$ and trained to predict future ES demands. This baseline uses a discrete-time recurrent neural network (RNN).
- **Transformer-driven demand prediction [31]:** A temporal Transformer-based predictor with self-attention is used over the historical context window. This serves as a powerful baseline, but typically requires more computing resources.

Performance evaluations consider the following metrics:

- **Accuracy:** We report the root mean square error (RMSE), mean absolute error (MAE), and symmetric mean absolute percentage error (sMAPE) of the H -step forecasts $\{\hat{N}_i^{\text{Dem},(v+1:v+H)}\}$ against the ground truth $\{N_i^{\text{Dem},(v+1:v+H)}\}$, averaged over all ESs and all test instances.

- **Model footprint:** We report the number of trainable parameters and the model size in megabytes using 32-bit floating-point (FP32) representation. These metrics characterize the storage and memory footprint of each predictor.

- **Robustness:** To assess robustness under realistic noisy conditions, we compare the mean squared error (MSE) on clean test inputs with that on corrupted inputs containing randomly missing observations, additive perturbations, and occasional demand bursts, and report the performance drop ΔMSE .

System-Level Benchmarks and Metrics: We then compare FUSION with several benchmark methods enumerated below:

- **Pure online scheduling (PurOnline) [16]:** This baseline removes the offline stage of FUSION: ESs and APs do not establish offline contracts, and UAV service routes are not pre-planned. HU/MU tasks are scheduled solely by PG-BRD using the currently available ES/AP resources.
- **Two-stage resource provisioning without PG-based online scheduling (FUSION_NoPG):** This variant keeps the same offline stage as FUSION, i.e., Off-AIC² with eACO-VRP is still

used to form offline AP-ES contracts and pre-planned routes. However, the online stage abandons PG-BRD: HU/MU tasks are randomly assigned to feasible ESs/APs within coverage subject to resource capacity and deadline constraints.

- **Two-stage resource provisioning with random offline contracts (FUSION_Random):** This baseline uses the same online stage as FUSION and still relies on PG-BRD for congestion-aware online resource scheduling. However, its offline stage no longer uses Off-AIC² or eACO-VRP; instead, each AP is randomly associated with a subset of ESs and moves along a random feasible route until its capacity is exhausted.

- **Two-stage resource provisioning without ES spatio-temporal features (FUSION_NoST):** This baseline preserves the two-stage structure of FUSION, but its offline auction ignores ES-specific time windows and travel-time constraints. Specifically, AP-ES pairs are determined only by simple price/volume trading without using eACO-VRP, while the online stage still uses PG-BRD for task scheduling.

To assess performance, we use the following metrics:

- **Social welfare (SW):** SW combines the offline welfare U^{SW} in (4) and the online welfare $\mathcal{W}(\pi)$ defined in Sec. V-B.
- **Delay incurred by interactions between SPs and SDs (DoI):** DoI captures the total decision-making latency (in milliseconds) from control-message exchanges between ESs/APs (SPs) and HUs/MUs (SDs) across both offline and online stages. To compute DoI, following [32], [34], [35], we assume that the per-interaction delay lies in the range of [1, 15] ms.
- **Energy consumption incurred by interactions between SPs and SDs (ECoI):** ECoI measures signaling-related energy consumption during decision making. Following [32], [34], the ES transmit power is set within [6, 20] W. Specifically, ECoI is computed by summing the energy consumed by all control-message transmissions, with interaction delays used as the transmission durations.
- **Near-truthfulness and individual rationality of Off-AIC²:** We examine participant utilities under unilateral bid/ask deviations, with utilities evaluated using true valuations and costs. The deviation tests assess the incentive to misreport, while comparisons between true costs/values and final rewards/payments assess individual rationality for winning participants.

B. Performance Evaluation on Prediction

1) *Robustness vs. Cost:* Table II compares prediction error and model size under clean and corrupted inputs. The corrupted setting includes 30% randomly missing values, Gaussian noise with standard deviation $\sigma = 0.05$, and burst shocks of length 1–3. On clean data, LSTM achieves the lowest MSE of 0.119, compared with 0.467 for Transformer and 0.459 for Pro-LNN. Pro-LNN uses 11.1k parameters, whereas LSTM uses 52.8k. Under corrupted inputs, the MSE increases to 0.983 for LSTM, 0.781 for Transformer, and 0.762 for Pro-LNN. Thus, Pro-LNN has a smaller model footprint and lower prediction error under the tested corruptions, although LSTM is more accurate on clean inputs.

2) *Real-World Dataset Test:* We evaluate Pro-LNN on the UCI Electricity Load Diagrams 2011–2014 dataset [36] using two load series, MT_005 and MT_145. In this univariate forecasting task, a history of $T_{\text{his}} = 48$ measurements is used

TABLE II
ROBUSTNESS–COMPLEXITY COMPARISON OF DIFFERENT FORECASTERS ON SYNTHETIC NON-STATIONARY SERIES. THE “CORRUPT” CASE INJECTS 30% RANDOM MISSING VALUES, GAUSSIAN NOISE WITH STANDARD DEVIATION $\sigma = 0.05$, AND RANDOM BURST SHOCKS.

Model	#Params	MSE _{clean}	MSE _{corrupt}	Δ MSE	Rel. increase
LSTM	52800	0.119	0.983	0.864	+726.3%
Transformer	69500	0.467	0.781	0.314	+67.3%
Pro-LNN	11100	0.459	0.762	0.303	+66.1%

TABLE III
PREDICTION ACCURACY AND MODEL FOOTPRINT ON THE UCI DATASET (LOAD SERIES MT_005 AND MT_145, INPUT LENGTH $T_{\text{his}} = 48$, PREDICTION HORIZON $H = 10$).

Model	#Params	Size (MB)	RMSE	MAE	sMAPE
LSTM	52842	0.204	0.1121	0.0816	0.1831
Transformer	69482	0.400	0.1158	0.0824	0.1864
Pro-LNN	11114	0.046	0.1234	0.0906	0.1983

TABLE IV
ROBUSTNESS TO MISSING AND NOISY INPUTS ON UCI ELECTRICITY (30% RANDOM MISSING VALUES, ADDED GAUSSIAN NOISE WITH STANDARD DEVIATION OF $\sigma = 0.05$, AND RANDOM BURST SHOCKS).

Model	MSE (clean)	MSE (corrupt)	Δ MSE
LSTM	0.0126	0.0485	0.0359
Transformer	0.0134	0.0592	0.0458
Pro-LNN	0.0152	0.0467	0.0315

to predict the next $H = 10$ time steps. The electricity-load traces serve as temporal proxies for ES-side demand, allowing us to evaluate forecasting performance on real-world load variations. Quantitative results are summarized in Table III and Table IV. As shown in Table III, Pro-LNN achieves comparable prediction accuracy to LSTM and Transformer, while being much more compact. In particular, Pro-LNN attains an RMSE of 0.1234, close to that of the LSTM (0.1121) and the Transformer (0.1158), but only uses 11,114 parameters and has a 0.046 MB storage/memory footprint. The parameter counts and model sizes indicate a smaller storage footprint for Pro-LNN, which is relevant to deployment on resource-constrained edge platforms. To evaluate robustness, we inject corrupted inputs, following the same configuration as in the synthetic experiments detailed in Sec. VI-B1. As reported in Table IV, Pro-LNN exhibits the smallest performance degradation among all models: its MSE increases from 0.0152 (clean) to 0.0467 (corrupt), yielding a Δ MSE of 0.0315, which is lower than that of both LSTM (0.0359) and Transformer (0.0458). These observations indicate that Pro-LNN is highly parameter-efficient and demonstrates strong robustness to missing data and measurement noise, which in turn supports the suitability of Pro-LNN for forecasting future ES resource demands in the offline stage of FUSION.

C. Performance Evaluation on FUSION

To emulate heterogeneous, time-varying online demands across different ESs, we adopt the real-world UCI Electricity Load Diagrams dataset [36], select several distinct load profiles,

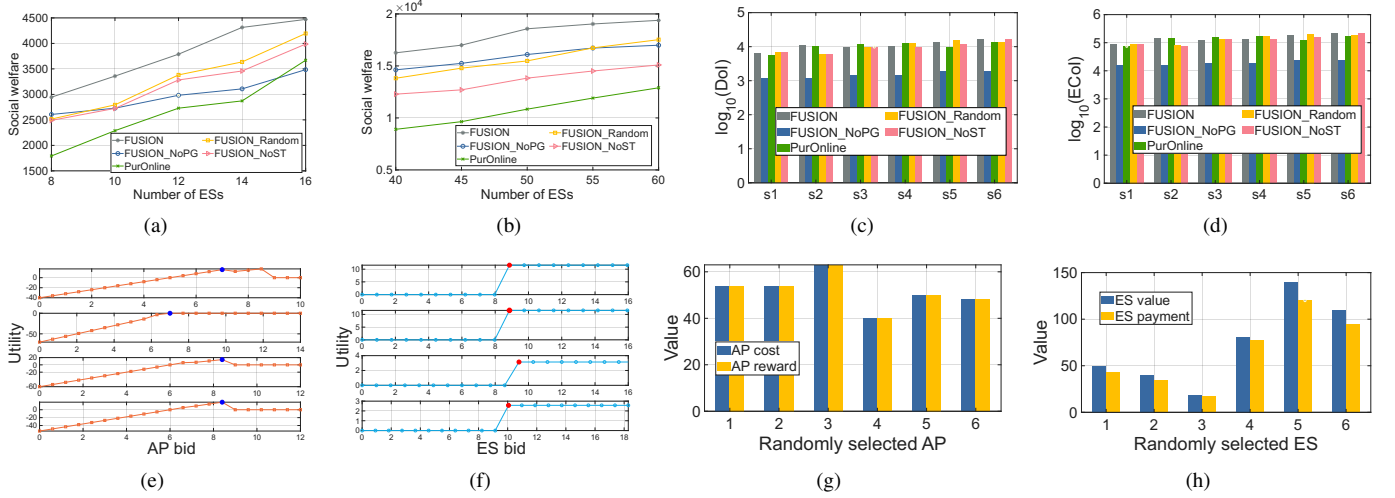


Fig. 2. Performance comparisons and economic property analyses: (a)–(b) SW under different problem sizes (80 HUs, 80 MUs, and 5 APs in (a), and 400 HUs, 400 MUs, and 35 APs in (b)); (c)–(d) DoI and ECoI under different problem sizes (s1–s6 are set as {75/75/15/8}, {75/75/20/8}, {90/90/20/10}, {90/90/25/10}, {115/115/25/12}, and {115/115/30/12}); (e)–(f) near-truthfulness; and (g)–(h) individual rationality.

and map their normalized power-consumption traces to the per-time-slot resource demand of each ES. We then train Pro-LNN on these profiles to forecast the per-ES demand¹⁰, and then evaluate FUSION as follows (i.e., in Sec. VI-C1–Sec. VI-C3).

1) *Social Welfare (SW)*: Figs. 2(a)–2(b) report SW as the number of ESs increases from 8 to 16. They correspond to two representative network configurations: 80 HUs, 80 MUs, and 5 APs in Fig. 2(a), and 400 HUs, 400 MUs, and 35 APs in Fig. 2(b) to capture different network scales. In Fig. 2(a), SW increases for all methods with more ESs, since they provide more computing resources and service opportunities. However, the growth rates differ across schemes. FUSION consistently achieves the highest SW because its offline stage employs Off-AIC² to match AP resources with ESs of high revenue potential under spatio-temporal constraints, while its online stage adopts a PG-BRD scheduler to perform congestion-aware task allocation among HUs and MUs, supporting more effective use of the available resources. PurOnline, which lacks an offline stage and relies solely on an online potential game over real-time ES resources, achieves the lowest SW due to the absence of proactive UAV repositioning. FUSION_NoPG shares the same offline design as FUSION but replaces PG-BRD with random online assignment, resulting in unresolved congestion and a notable SW degradation, highlighting the importance of the online PG. FUSION_Random adopts random AP–ES association in the offline stage while retaining PG-BRD online, yielding intermediate performance and indicating that intelligent online scheduling alone cannot compensate for the lack of revenue- and spatio-temporal-aware offline planning. FUSION_NoST ignores ES spatio-temporal characteristics in the offline stage and relies on a price-driven auction with PG-BRD online; it outperforms PurOnline and FUSION_Random

but still exhibits a consistent gap to FUSION. Fig. 2(b) shows qualitatively similar trends under the larger-scale configuration: when ESs increase, SW rises for all methods, with FUSION consistently outperforming benchmarks. Results show that offline route-aware contracting and online congestion-aware scheduling both contribute to SW gains across tested network scales.

2) *Evaluation on Interaction Overhead*: To quantitatively assess interaction efficiency, we analyze DoI (Fig. 2(c)) and ECoI (Fig. 2(d)). The x-axis (s1–s6) denotes different system configurations (various HU/MU/ES/AP combinations detailed in the caption of the figure), and the y-axis uses a logarithmic scale to capture results across magnitudes. Across all settings, DoI and ECoI remain within the same order of magnitude: FUSION is not always the best in DoI/ECoI, but the gaps with baselines (FUSION_NoPG, FUSION_Random, FUSION_NoST, PurOnline) are rather small. This arises from FUSION’s offline contracting stage, which exposes more candidate SPs and resource options, prompting the PG-BRD online scheduler to explore a larger strategy space, slightly increasing interaction rounds and per-transaction overhead. Auction and route planning are completed before online service execution. Their amortized overhead is included in the DoI and ECoI reported in Figs. 2(c)–2(d). Combined with SW results (Figs. 2(a)–2(b)), we conclude that FUSION maintains comparable DoI/ECoI to baselines while substantially improving SW through more efficient air–ground resource utilization.

3) *Near-Truthfulness and Individual Rationality*: We evaluate the near-truthfulness and individual rationality of Off-AIC² with 75 HUs, 75 MUs, 12 ESs, and 7 APs. Figs. 2(e) and 2(f) plot seller (AP) and buyer (ES) utilities against reported asks/bids, respectively, while Figs. 2(g) and 2(h) verify individual rationality by comparing true costs/values with payments. Fig. 2(e) shows four independent market instances, each with one randomly selected AP. The horizontal axis denotes the reported ask and the vertical axis the resulting utility (revenue minus true cost), with the blue dot marking the true ask. As the ask increases, the AP’s utility rises while it remains

¹⁰We employ the UCI dataset as a trace to capture realistic temporal patterns of aggregate demand at each ES. Pro-LNN is trained on these time series and used to forecast the demand $N_i^{\text{Dem},(v)}$ of ES s_i over future trading rounds; the forecasted demand profiles drive the time-varying SD task-arrival intensities at each ES, while the total population sizes of SDs and SPs are kept fixed to control the overall network scale.

selected, then drops sharply beyond its critical region when the contract is lost. Across the instances, utility is maximized near the true ask, indicating near-truthful behavior. Fig. 2(f) presents the corresponding ES results. Each subplot selects one ES, with the horizontal axis representing the reported mean bid and the red dot indicating the true valuation. Below a threshold close to the true value, the ES obtains zero utility; after crossing it, the ES wins, while further bid increases yield negligible gains. Thus, the best-response region consistently concentrates around the true valuation. Step 3 of Off-AIC² approximates critical-value payments and rewards through binary search, and the utility curves reflect the resulting pricing procedure and route-aware allocation produced by eACO-VRP. Appx. B.2 states the bounded-error assumptions used in the near-truthfulness analysis. The negative utilities in parts of Figs. 2(e) and 2(f) arise from hypothetical reports evaluated using true valuations and costs. Figs. 2(g) and 2(h) further confirm individual rationality: for randomly selected winning APs, the total reward determined by Off-AIC² is no smaller than the corresponding true total cost, while for selected ESs, the true valuation always exceeds the final payment.

4) *Additional DRL Comparison and Decomposition Gap:* Appx. F further compares PG-BRD with representative DRL-based online schedulers, including DQN, PPO, and MADDPG, under identical offline service preparation. The results show that PG-BRD achieves higher social welfare and timely-service performance, whereas the DRL-based variants exhibit advantages in served ratio, energy cost, and online runtime. Appx. G evaluates the two-stage FUSION framework against small-scale full-information benchmarks. The results show that FUSION maintains a limited performance gap relative to the exact solutions or solver upper bounds while requiring substantially lower runtime, demonstrating a favorable performance-complexity tradeoff in the tested instances.

VII. CONCLUSION AND FUTURE WORK

This work proposed FUSION, a forecasting-driven two-stage framework for service provisioning in air-ground integrated networks with human-machine coexistence. The offline stage combines Pro-LNN demand forecasting, eACO-VRP route planning, and Off-AIC² contracting to prepare AP-assisted service resources. The online stage uses PG-BRD to update SD-SP assignments under congestion and capacity constraints. When the convergence condition is satisfied, PG-BRD returns an ε -NE, which becomes a pure-strategy NE when $\varepsilon = 0$. Experiments show higher social welfare with interaction delay and signaling energy overheads comparable to the considered baselines. Future work will investigate event-triggered AP replanning and adaptive service coordination under abrupt demand, mobility, and topology changes.

REFERENCES

- [1] L. Kong, J. Tan, J. Huang, G. Chen, S. Wang, X. Jin, P. Zeng, M. Khan, and S. K. Das, "Edge-computing-driven Internet of Things: A survey," *ACM Comput. Surv.*, vol. 55, no. 8, Art. no. 174, pp. 1-41, Dec. 2023.
- [2] Z. Ning, H. Hu, X. Wang, L. Guo, S. Guo, G. Wang, and X. Gao, "Mobile edge computing and machine learning in the Internet of unmanned aerial vehicles: A survey," *ACM Comput. Surv.*, vol. 56, no. 1, Art. no. 13, pp. 1-31, Aug. 2024.
- [3] H. Qi, M. Liwang, X. Wang, L. Li, W. Gong, J. Jin, and Z. Jiao, "Bridge the Present and Future: A Cross-Layer Matching Game in Dynamic Cloud-Aided Mobile Edge Networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 12522-12539, Dec. 2024.
- [4] C. Feng, M. Yang, Z. Jing, T. Q. S. Quek, and M. Mei, "Latency-Aware Service Deployment and Peer Offloading: A Long-Term Optimization Framework for Satellite Edge Computing," *IEEE Internet Things J.*, vol. 13, no. 1, pp. 405-417, Jan., 2026.
- [5] M. Qin, N. Cheng, Z. Jing, T. Yang, W. Xu, Q. Yang, and R. R. Rao, "Service-Oriented Energy-Latency Tradeoff for IoT Task Partial Offloading in MEC-Enhanced Multi-RAT Networks," *IEEE Internet Things J.*, vol. 8, no. 3, pp. 1896-1907, Feb., 2021.
- [6] G. Sun, Y. Wang, Z. Sun, Q. Wu, J. Kang, D. Niyato, and V. C. M. Leung, "Multi-objective optimization for multi-UAV-assisted mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 14803-14820, 2024.
- [7] M. Hui, J. Chen, L. Yang, L. Lv, H. Jiang, and N. Al-Dhahir, "UAV-assisted mobile edge computing: Optimal design of UAV altitude and task offloading," *IEEE Trans. Wireless Commun.*, vol. 23, no. 10, pp. 13633-13647, Oct. 2024.
- [8] Z. Jia, J. He, Y. Cui, Q. Zhu, L. Yuan, F. Zhou, Q. Wu, D. Niyato, and Z. Han, "Hierarchical low-altitude wireless network empowered air traffic management," *IEEE Commun. Mag.*, early access, 2026.
- [9] Z. Jia, J. He, L. He, M. Sheng, J. Liu, Q. Wu, and Z. Han, "Dynamic trajectory optimization and power control for hierarchical UAV swarms in 6G aerial access network," *IEEE Trans. Wireless Commun.*, vol. 25, pp. 3349-3362, 2026.
- [10] X. Wang, J. Li, J. Wu, L. Guo, and Z. Ning, "Energy efficiency optimization of IRS and UAV-assisted wireless powered edge networks," *IEEE J. Sel. Topics Signal Process.*, vol. 18, no. 7, pp. 1297-1310, 2024.
- [11] J. Xu, H. Yao, R. Zhang, T. Mai, and M. Guizani, "Low latency and accuracy-guaranteed DNN inference for UAV-assisted IoT networks," *IEEE Trans. Cogn. Commun. Netw.*, early access, 2025, doi: 10.1109/TCNN.2025.3542443.
- [12] Y. Li, J. Rao, L. Wang, Y. Xiao, X. Ge, and G. Shi, "Distributed optimization of resource efficiency for federated edge intelligence in AAV-enabled IoT networks," *IEEE Internet Things J.*, vol. 12, no. 8, pp. 9449-9464, Apr. 2025.
- [13] L. Zeng, H. Chen, D. Feng, X. Zhang, and X. Chen, "A3D: Adaptive, accurate, and autonomous navigation for edge-assisted drones," *IEEE/ACM Trans. Netw.*, vol. 32, no. 1, pp. 713-728, Feb. 2024.
- [14] Z. Sun, G. Sun, Q. Wu, L. He, S. Liang, H. Pan, D. Niyato, C. Yuen, and V. C. M. Leung, "TJCCT: A two-timescale approach for UAV-assisted mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 24, no. 4, pp. 3130-3147, Apr. 2025.
- [15] Y. Ding, K. Li, C. Liu, and K. Li, "A Potential Game Theoretic Approach to Computation Offloading Strategy Optimization in End-Edge-Cloud Computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 6, pp. 1503-1519, Jun. 2022.
- [16] Z. Zhou, L. Pan, and S. Liu, "Potential Game-Based Computation Offloading in Edge Computing With Heterogeneous Edge Servers," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 1, pp. 290-301, Jan. 2025.
- [17] C. Zhang, C. Lee, D. Niyato, and P. Wang, "Auction approaches for resource allocation in wireless systems: A survey," *IEEE Commun. Surveys Tuts.*, vol. 15, no. 3, pp. 1020-1041, 3rd Quart., 2013.
- [18] H. Ren, K. Liu, G. Yan, C. Liu, Y. Li, C. Li, and W. Wu, "Truthful auction mechanisms for dependent task offloading in vehicular edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 14987-15002, Dec. 2024.
- [19] X. Xu, Y. Hu, G. Cui, L. Qi, W. Dou, and Z. Cai, "CADEC: A combinatorial auction for dynamic distributed DNN inference scheduling in edge-cloud networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 10024-10041, Oct. 2025.
- [20] X. Li, K. Mo, Y. Hou, Z. Li, H. Xu, and N. Guan, "An online auction approach to computing resource allocation in mobile AIGC networks," *IEEE Internet Things J.*, vol. 12, no. 8, pp. 11182-11193, Apr. 2025.
- [21] R. Lu, W. Zhang, Y. Wang, Q. Li, X. Zhong, H. Yang, and D. Wang, "Auction-based cluster federated learning in mobile edge computing systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 4, pp. 1145-1158, Apr. 2023.
- [22] C.-Y. Hsieh, Y. Ren, and J.-C. Chen, "Edge-cloud offloading: Knapsack PG in 5G multi-access edge computing," *IEEE Trans. Wireless Commun.*, vol. 22, no. 11, pp. 7158-7171, Nov. 2023.
- [23] L. Wu, P. Sun, Z. Wang, Y. Li, and Y. Yang, "Computation offloading in multi-cell networks with collaborative edge-cloud computing: A game theoretic approach," *IEEE Trans. Mobile Comput.*, vol. 23, no. 3, pp. 2093-2106, Mar. 2024.

- [24] A. R. Hossain, W. Liu, and N. Ansari, "AI-native end-to-end network slicing for next-generation mission-critical services," *IEEE Trans. Cogn. Commun. Netw.*, vol. 11, no. 1, pp. 48-58, 2025.
- [25] S. Wu, M. Liwang, D. Wang, X. Wang, C. Wu, J. Tang, L. Li, and X. Xia, "Effective two-stage double auction for dynamic resource provision over edge networks via discovering the power of overbooking," *IEEE Trans. Serv. Comput.*, early access, pp. 1-13, 2025.
- [26] M. Chahine, R. Hasani, P. Kao, A. Ray, R. Shubert, M. Lechner, A. Amini, and D. Rus, "Robust flight navigation out of distribution with liquid neural networks," *Sci. Robot.*, vol. 8, no. 77, Art. eadc8892, 2023.
- [27] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," arXiv preprint arXiv:1607.06450, 2016.
- [28] H. Qi, M. Liwang, X. Wang, L. Fu, Y. Hong, L. Li, and Z. Cheng, "Accelerating stable matching between workers and spatial-temporal tasks for dynamic MCS: A stagewise service trading approach," *IEEE Trans. Mobile Comput.*, early access, pp. 1-16, 2025, doi: 10.1109/TMC.2025.3610915.
- [29] J. Li, Y. Xiong, and J. She, "UAV path planning for target coverage task in dynamic environment," *IEEE Internet Things J.*, vol. 10, no. 20, pp. 17734-17745, 2023.
- [30] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735-1780, Nov. 1997, doi: 10.1162/neco.1997.9.8.1735.
- [31] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017.
- [32] H. Qi, M. Liwang, S. Hosseinalipour, L. Fu, S. Zou, and W. Ni, "Future resource bank for ISAC: Achieving fast and stable win-win matching for both individuals and coalitions," *IEEE J. Sel. Areas Commun.*, early access, 2025, doi: 10.1109/JSAC.2025.3608114.
- [33] Z. Yang, S. Bi, and Y.-J. A. Zhang, "Dynamic offloading and trajectory control for UAV-enabled mobile edge computing system with energy harvesting devices," *IEEE Trans. Wireless Commun.*, vol. 21, no. 12, pp. 10515-10528, 2022.
- [34] 5G Americas, "New services & applications with 5G ultra-reliable low latency communications," White Paper, Nov. 2018.
- [35] 3GPP, "5G; Study on scenarios and requirements for next generation access technologies," 3GPP TR 38.913, ver. 17.0.0, Release 17, 2022.
- [36] A. Trindade, "ElectricityLoadDiagrams20112014," UCI Mach. Learn. Repository, 2015. [Online]. Available: <https://doi.org/10.24432/C58C86>.

APPENDIX A KEY NOTATIONS

Key notations in this paper are summarized in Table V.

APPENDIX B DETAILS OF OFFLINE STAGE

A. Detailed Pro-LNN Architecture and Training Procedure

In this section, we provide the detailed update equations and training procedure of the proposed Pro-LNN predictor used in Off-AIC².

Step 1. Stable liquid cell (line 7, Alg. 4): At each transition $\nu \rightarrow \nu + 1$, the liquid cell maps the new input $\mathbf{z}_i^{(\nu+1)}$ and the previous hidden state $\mathbf{h}_i^{(\nu)} \in \mathbb{R}^d$ to $\mathbf{h}_i^{(\nu+1)}$. Concretely, we first compute neuron-wise time constants and a pre-activation: $\tau^{(\nu)} = \text{softplus}(W_\tau[\mathbf{z}_i^{(\nu+1)}, \mathbf{h}_i^{(\nu)}]) + \tau_{\min}$ and $\tilde{\mathbf{u}}^{(\nu)} = W_h \mathbf{h}_i^{(\nu)} + W_x \mathbf{z}_i^{(\nu+1)} + \mathbf{b}$, where W_τ, W_h, W_x and $\mathbf{b}$ are trainable, and $\tau_{\min} > 0$ enforces strictly positive time scales (we also clamp $\tau^{(\nu)}$ to $[\tau_{\min}, \tau_{\max}]$ for numerical stability). The effective update rate is then $\alpha^{(\nu)} = \text{clip}\left(\frac{\Delta\nu}{\tau^{(\nu)}}, \alpha_{\min}, \alpha_{\max}\right)$, with a fixed $\Delta\nu$ (one trading interval in our implementation) and small constants $0 < \alpha_{\min} \leq \alpha_{\max} < 1$. We then apply a lightweight layer normalization and non-linearity, $\tilde{\mathbf{h}}_i^{(\nu)} = \tanh(\text{LN}(\tilde{\mathbf{u}}^{(\nu)}))$, and update the hidden state as

$$\mathbf{h}_i^{(\nu+1)} = (1 - \alpha^{(\nu)}) \odot \mathbf{h}_i^{(\nu)} + \alpha^{(\nu)} \odot \tilde{\mathbf{h}}_i^{(\nu)} + \gamma \tanh(\text{LN}(\mathbf{h}_i^{(\nu)})), \quad (25)$$

where γ is a small residual scaling factor and $\odot$ denotes element-wise multiplication. This leaky-integrator update blends exponential memory decay with input-driven state injection, while the neuron-wise $\tau^{(\nu)}$ allow different neurons to operate at different effective time scales.

Notation. For clarity, all nonlinear operators in the above update are applied element-wise unless otherwise stated. Specifically, $\text{softplus}(\cdot)$ denotes the standard softplus activation $\text{softplus}(x) = \log(1 + e^x)$, $\text{clip}(x, a, b)$ clips each element of x into the interval $[a, b]$, $\text{LN}(\cdot)$ denotes layer normalization [27], and $\text{LiquidCell}(\mathbf{z}_i^{(\nu+1)}, \mathbf{h}_i^{(\nu)})$ is a shorthand for one step of the stable liquid update in Step 1 (using the time-constant gating and residual integration described above).

Step 2. Historical encoding (lines 5-7, Alg. 4): Given the latest L observations of ES s_i up to trading round ν , Pro-LNN encodes the history by iteratively applying the liquid cell: $\mathbf{h}_i^{(\nu-L+1)} \leftarrow \text{LiquidCell}(\mathbf{z}_i^{(\nu-L+1)}, \mathbf{0}), \dots, \mathbf{h}_i^{(\nu)} \leftarrow \text{LiquidCell}(\mathbf{z}_i^{(\nu)}, \mathbf{h}_i^{(\nu-1)})$.

The neuron-wise time constants control memory retention and state updates for non-stationary or noisy demand trajectories.

Step 3. Multi-horizon readout and consistency (lines 8-17, Alg. 4): After encoding the history, a shallow MLP readout maps the final hidden state to an H -step forecast:

$$\hat{\mathbf{n}}_i^{(1:H)} = \text{Readout}(\mathbf{h}_i^{(\nu)}) = [\hat{N}_i^{\text{Dem},(\nu+1)}, \dots, \hat{N}_i^{\text{Dem},(\nu+H)}], \quad (26)$$

where $\text{Readout}(\cdot)$ denotes a lightweight fully connected network that maps the d -dimensional hidden state to an H -dimensional

TABLE V
KEY NOTATIONS

Notation	Explanation
$\mathcal{S}, \mathcal{U}^H, \mathcal{U}^M, \mathcal{V}$	Sets of edge servers (ESs), human users (HUs), machine users (MUs), and vehicle-UAV agent pairs (APs)
s_i, u_m^H, u_n^M, v_k	The i -th ES, the m -th HU, the n -th MU, and the k -th AP
$N_i^{\text{Dem},(\nu)}, \hat{N}_i^{\text{Dem},(\nu+h)}$	Actual and LNN-predicted task demand of ES s_i at trading rounds ν and $\nu+h$
$l_i^E, l_k^A(t)$	Location of ES s_i and ground location of AP v_k in time t
r_m^H, r_n^M, r_j	Computation workload (CPU cycles) of HU u_m^H , MU u_n^M , and generic SD u_j^{On}
d_m^H, d_n^M, d_j	Input data size (bits) of HU tasks, MU tasks, and generic SD tasks
$f_i^E, f_i^{\text{On}}, f_m^{\text{H,loc}}, f_n^{\text{M,loc}}, f_k^A$	Computing capabilities of ES s_i , online SP s_i^{On} , local HU/MU devices, and UAVs dispatched by AP v_k
$e_m^{\text{H,tx}}, e_n^{\text{M,tx}}$	Uplink transmit power of HU u_m^H and MU u_n^M
e_i^E	Local computing power consumption of ES s_i
$e_k^{\text{comp}}, e_k^{\text{move}}, e_k^{\text{fly}}$	Power consumption associated with UAV computation, ground movement, and UAV flight for AP v_k
$\tau_n^{\text{ddl}}, \tau_j^{\text{ddl}}$	Task deadline of an MU, indexed by n in the MU model and by j in the online formulation
C_k^A, G_k^A, Q_k^A	Number of UAVs carried by AP v_k , maximum number of SDs served by each UAV, and total AP service capacity
$N_{i,k}^{\text{Trad}}$	Quantity of traded computing resources between ES s_i and AP v_k in an offline contract
$x_{i,k}^{\text{Off}}, \mathbf{X}^{\text{Off}}$	Binary offline assignment between ES s_i and AP v_k ; $\mathbf{X}^{\text{Off}}$ collects all $x_{i,k}^{\text{Off}}$
$p_{i,k}^{\text{ES}}, r_{i,k}^{\text{ES}}, \bar{p}_i$	Unit payment collected from ES s_i , unit reward paid to AP v_k , and historical average user service price at ES s_i
$U_{i,k}^{\text{ES,Off}}, U_k^{\text{A,Off}}, U^{\text{Auc,Off}}$	Utility of ES s_i , utility of AP v_k , and utility of the auctioneer in the offline stage
U^{SW}	Total SW of the offline auction market (sum of utilities of ESs, APs, and auctioneer)
$\mathcal{G}_k(\mathcal{N}_k, \mathcal{E}_k), \mathbb{K}$	Directed complete graph and ant set used by eACO-VRP for routing AP v_k
$\varphi_{r,s}, \eta_{r,s}, pr_{r,s}^k, \Delta U_{r,s}^A$	Pheromone level, distance-based heuristic desirability, transition probability of ant k , and AP-side utility increment on edge (n_r, n_s) in eACO-VRP
$\mathcal{S}^{\text{On}}, \mathcal{U}^{\text{On}}, \mathcal{U}^H, \mathcal{U}^M$	Sets of online service providers (SPs), online service demanders (SDs), HUs, and MUs
$x_{j,0}^{\text{On}}, x_{j,i}^{\text{On}}, \mathbf{X}_j^{\text{On}}$	Indicator of local execution, indicator of scheduling SD u_j^{On} to SP s_i^{On} , and the one-hot scheduling vector of SD u_j^{On}
$y_i, \mathbb{x}_i, n_i(\pi)$	Effective congestion load, aggregate computing load, and number of associated SDs at SP s_i^{On} under profile π
$t_{i,i}^{\text{comp}}, t_{i,i}^{\text{tx}}, T_{i,i}^{\text{edge}}$	Computation delay, uplink transmission delay, and total edge-side latency of SD u_i^{On} at SP s_i^{On}
$S_i(\hat{\mathbf{a}}), \kappa_j, \theta_{j,i}, \tau_i(y)$	Non-congestion benefit term, congestion sensitivity, load factor, and congestion function at SP s_i^{On}
$K_i(\mathbb{x}_i), G_i$	Computation/operation cost function and maximum number of SDs that online SP s_i^{On} can concurrently serve
$\pi, \Phi^G(\pi), \mathcal{F}^{(I)}, \mathcal{F}^{(II)}$	Joint scheduling profile, game potential function, and optimization problems in the offline and online stages

Algorithm 4: Pro-LNN

Input: Training sequences $\{z_i^{(1:T)}\}$; history length L ; horizon H ; learning rate η ; consistency depth K

Output: Predicted sequences $\{\hat{N}_i^{\text{Dem},(\nu+1:\nu+H)}\}$

```

1 Initialize liquid-cell and readout parameters  $\theta$ ;
2 while not converged do
3   Sample a mini-batch  $\mathcal{B}$  of training windows from  $\mathcal{D}$ ;
4   for each window  $(z_i^{(\nu-L+1:\nu)}, N_i^{\text{Dem},(\nu+1:\nu+H)}) \in \mathcal{B}$  do
5     Set hidden state  $\mathbf{h}_i \leftarrow \mathbf{0}$ ;
6     for  $k = \nu - L + 1$  to  $\nu$  do
7        $\mathbf{h}_i \leftarrow \text{LiquidCell}(z_i^{(k)}, \mathbf{h}_i)$ ;
8     (Base  $H$ -step forecast)
9      $\hat{\mathbf{n}}_i^{(1:H)} \leftarrow \text{Readout}(\mathbf{h}_i)$ ;
10     $\mathcal{L}_i \leftarrow \sum_{h=1}^H (\hat{n}_i^{(h)} - N_i^{\text{Dem},(\nu+h)})^2$ ;
11    if  $K > 0$  then
12      Construct a history window  $\mathbf{y}_i$  from
         $\{z_i^{(\nu-L+1)}, \dots, z_i^{(\nu)}\}$ ;
13      for  $k = 1$  to  $K$  do
14         $\hat{\mathbf{n}}_i^{(1:H)} \leftarrow f_\theta(\mathbf{y}_i)$ ;
15         $\mathcal{L}_i \leftarrow \mathcal{L}_i + (\hat{n}_i^{(1)} - N_i^{\text{Dem},(\nu+k)})^2$ ;
16        Update  $\mathbf{y}_i$  by discarding the oldest step and
          appending  $\hat{n}_i^{(1)}$  as the newest demand;
17     $\mathcal{L}_i \leftarrow \mathcal{L}_i / (H + K)$ ;
18  Aggregate loss  $\mathcal{L} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \mathcal{L}_i + \lambda \Omega(\theta)$ ;
19  Update parameters  $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$ ;
```

prediction vector. The base training loss is the mean squared error (MSE) over the H horizons. To further improve stability for long horizons, we optionally add a *multi-step consistency* objective: starting from a history window, we repeatedly roll the model forward in closed loop by feeding its own one-step predictions back into the window, and penalize the mismatch at each rolled-out step.

Step 4. Inference and integration : At inference, Pro-LNN takes, for each ES s_i , the latest L observations $\{z_i^{(\nu-L+1)}, \dots, z_i^{(\nu)}\}$, encodes them via the liquid cell into $\mathbf{h}_i^{(\nu)}$, and applies the readout to obtain an H -step forecast $\{\hat{N}_i^{\text{Dem},(\nu+1)}, \dots, \hat{N}_i^{\text{Dem},(\nu+H)}\}$. These predictions are computed in the offline stage before each trading round and then fed into: (i) the eACO-VRP module for time-window-aware ES recommendation and AP route pre-planning, and (ii) the auction module for ES-AP contract design. Pro-LNN inference is completed during offline service preparation, before the corresponding online service window begins.

B. Economic Properties of Off-AIC²

We analyze the economic properties of Off-AIC² under standard mechanism-design assumptions. Specifically, participants have quasi-linear utilities; each ES has an independent private valuation for receiving service, and each AP has an independent private cost for providing service; participants do not collude; deterministic tie-breaking is used whenever multiple allocations have the same score; and service-capacity/window information is verifiable by the auction platform. In addition, the final settlement of each active ES-AP contract is selected within a non-deficit settlement band, as follows:

$$\text{ask}_k \leq r_{i,k}^{\text{ES}} \leq p_{i,k}^{\text{ES}} \leq \overline{\text{bid}}_i, \quad \forall (i, k) \text{ with } x_{i,k}^{\text{Off}} = 1, \quad (27)$$

which ensures that the ES payment is sufficient to cover the AP reward while remaining no larger than the ES-side declared valuation. These assumptions define the applicability scope of the individual-rationality, ϵ -near-truthfulness, and weak-budget-balance results established below.

Definition 1 (Individual Rationality). *Off-AIC² is individually rational if every winning ES obtains nonnegative utility, i.e., it pays no more than its true valuation for the contracted service, and every winning AP obtains nonnegative utility, i.e., it receives a reward no lower than its true service cost.*

Definition 2 (ϵ -Near-Truthfulness). *Let q denote any participant in the offline ES-AP contracting mechanism, with true type θ_q , reported type $\hat{\theta}_q$, and other participants' reports $\mathbf{b}_{-q}$. Let $U_q(\theta_q; \hat{\theta}_q, \mathbf{b}_{-q})$ denote the utility of participant q when its true type is θ_q but it reports $\hat{\theta}_q$. Off-AIC² is ϵ -near-truthful if, for any participant q and any possible misreport $\hat{\theta}_q \in \Theta_q$,*

$$U_q(\theta_q; \theta_q, \mathbf{b}_{-q}) \geq \sup_{\hat{\theta}_q \in \Theta_q} U_q(\theta_q; \hat{\theta}_q, \mathbf{b}_{-q}) - \epsilon. \quad (28)$$

This means that the maximum utility improvement obtainable through unilateral misreporting is bounded by ϵ .

Definition 3 (Weak Budget Balance). *Off-AIC² is weakly budget balanced if the total payment collected from all winning ESs is no smaller than the total reward paid to all winning APs. Equivalently, the auctioneer does not incur a deficit.*

Definition 4 (Bounded Approximation Error). *In an exactly monotone single-parameter allocation rule, raising an ES's bid should not reduce its chance of winning, and lowering an AP's ask should not reduce its chance of winning. In Off-AIC², the route-aware eACO-VRP subroutine may introduce small non-monotone perturbations because ES-AP matching is coupled with route feasibility, service-window compatibility, AP capacity, and travel costs. For each participant q , let Δ_q^{alloc} denote the maximum utility gain that can be caused by such route-induced non-monotone allocation perturbations, and let η_q^{price} denote the maximum utility gain caused by binary-search approximation in the critical-value-like pricing step. We assume that*

$$\Delta_q^{\text{alloc}} + \eta_q^{\text{price}} \leq \epsilon_q, \quad \epsilon = \max_q \epsilon_q. \quad (29)$$

We next prove that Off-AIC² satisfies individual rationality, ϵ -near-truthfulness, and weak budget balance under the above assumptions.

Proposition 2 (Individual Rationality of Off-AIC²). *All winning ESs and APs obtained from Off-AIC² are individually rational in the offline stage.*

Proof. The individual rationality of Off-AIC² follows from the candidate-pair screening rule in Step 2 and the settlement rule in Step 3 of Alg. 2. We prove it separately for the ES side and the AP side.

(i) **ES side.** Consider a winning ES s_i matched with AP v_k . In Step 2 of Alg. 2, for each AP v_k , the candidate ES set is constructed as

$$C_k = \{s_i \in \mathcal{U}^{\text{free}} \mid U_{i,k}^{\text{Off}} > 0, \overline{\text{bid}}_i \geq \text{ask}_k\}, \quad (30)$$

where $U_{i,k}^{\text{Off}} = \overline{\text{bid}}_i - \text{ask}_k$ denotes the pairwise surplus. Hence, if s_i is finally matched with v_k , the pair must satisfy $\overline{\text{bid}}_i \geq \text{ask}_k$ and has a nonempty settlement band.

Let v_i denote the true per-unit valuation of ES s_i . Under truthful reporting, $v_i = \overline{\text{bid}}_i$. By the settlement rule in (27), the final ES-side payment satisfies

$$p_{i,k}^{\text{ES}} \leq \overline{\text{bid}}_i = v_i. \quad (31)$$

Therefore, the utility of winning ES s_i is

$$U_i^{\text{ES,Off}} = N_{i,k}^{\text{Trad}}(v_i - p_{i,k}^{\text{ES}}) \geq 0. \quad (32)$$

Thus, every winning ES obtains nonnegative utility.

(ii) AP side. Consider a winning AP v_k . In Step 2, AP v_k can be matched only with ESs that satisfy the surplus and feasibility conditions, and its total assigned demand must obey the capacity constraint

$$\sum_i x_{i,k}^{\text{Off}} D_i \leq Q_k^{\text{A}}. \quad (33)$$

Let c_k denote the true per-unit service cost of AP v_k . Under truthful reporting, $c_k = \text{ask}_k$. By the settlement rule in (27), for every active pair (i, k) , the AP-side reward satisfies

$$r_{i,k}^{\text{ES}} \geq \text{ask}_k = c_k. \quad (34)$$

Hence, the utility of AP v_k is

$$U_k^{\text{A,Off}} = \sum_i x_{i,k}^{\text{Off}} N_{i,k}^{\text{Trad}}(r_{i,k}^{\text{ES}} - c_k) \geq 0. \quad (35)$$

Therefore, every winning AP obtains nonnegative utility. Combining the ES and AP sides, Off-AIC² is individually rational in the offline stage. ■

Proposition 3 (ϵ -Near-Truthfulness of Off-AIC²). *Under quasi-linear utilities, independent private values/costs, deterministic tie-breaking, and bounded approximation error, Off-AIC² is ϵ -near-truthful for both ESs and APs in the offline stage.*

Proof. We first clarify the role of ϵ . If the ES-AP allocation rule were exactly monotone and the payment/reward were computed as the exact critical value, the standard single-parameter auction argument would imply dominant-strategy truthfulness. In Off-AIC², however, ES-AP allocation is coupled with route feasibility, AP capacity, service windows, and the eACO-VRP route-construction subroutine. These factors may introduce small deviations from exact monotonicity. In addition, the critical-value-like payment and reward are obtained through binary search and may contain a small approximation error. By Definition 4, the maximum utility gain caused by these two effects is bounded by ϵ_q for participant q .

At a high level, Off-AIC² follows the same structural principle as classical truthful single-parameter auctions: it combines an approximately monotone allocation rule with critical-value-like payments. We prove the near-truthfulness property for ES buyers and AP sellers separately.

(i) ES side. Fix any ES s_i and treat all other bids and asks as given. Under the single-parameter abstraction, s_i reports a per-unit bid b_i , which determines its average bid $\overline{\text{bid}}_i = b_i$ and its pairwise surplus values with APs.

Approximate monotone allocation. In Step 1, ESs are sorted in nonincreasing order of $\text{bid}_i = b_i$. Increasing b_i can only move s_i upward in the sorted list and therefore cannot worsen its admission rank. Since Off-AIC² admits the top- I^* ESs into the subsequent matching stage, once s_i belongs to the admitted ES set at bid b_i , it remains admitted for any higher bid $b'_i > b_i$.

In Step 2, s_i can be matched with AP v_k only if

$$U_{i,k}^{\text{Off}} = b_i - \text{ask}_k > 0, \quad b_i \geq \text{ask}_k. \quad (36)$$

Thus, increasing b_i weakly increases all pairwise surplus values involving s_i , making s_i no less attractive in the candidate matching stage. If the route-aware allocation were exactly monotone, this would imply exact ES-side monotonicity. With eACO-VRP, the selected ES set may be slightly perturbed by route feasibility, service-window compatibility, travel cost, and AP capacity. By the bounded approximation-error definition, any additional utility gain caused by such route-induced non-monotonicity and pricing approximation is at most ϵ_i .

Critical-value-like payment. For each winning ES s_i , Step 3 treats its effective bid as a variable and performs binary search over the feasible interval to find the smallest effective bid that keeps s_i in the winning set. The resulting value is used to determine the payment $p_{i,k}^{\text{ES}}$ within the settlement band. Under exact monotonicity and exact critical-value pricing, this payment would eliminate profitable unilateral bid deviations. Under approximate monotonicity and finite-precision binary search, all remaining deviation gains are bounded by ϵ_i . Therefore, for any misreported bid $\hat{b}_i$,

$$U_i(b_i; b_i, \mathbf{b}_{-i}) \geq U_i(b_i; \hat{b}_i, \mathbf{b}_{-i}) - \epsilon_i. \quad (37)$$

(ii) AP side. Now fix any AP v_k and treat all ES bids and other APs' asks as given. Under the single-parameter abstraction, v_k reports a per-unit ask a_k .

Approximate monotone allocation. In Step 1, APs are sorted in nondecreasing order of $\text{ask}_k = a_k$. Lowering a_k can only move v_k upward in the sorted list and therefore cannot worsen its admission rank. Since Off-AIC² admits the top- K^* APs into the matching stage, once v_k is admitted at ask a_k , it remains admitted for any lower ask $a'_k < a_k$.

In Step 2, v_k 's competitiveness is determined by the pairwise surplus

$$U_{i,k}^{\text{Off}} = \overline{\text{bid}}_i - a_k. \quad (38)$$

Lowering a_k weakly increases all surplus values involving v_k , making it no less competitive in the matching stage. If the allocation were exactly monotone, lowering the ask would not reduce the chance that v_k wins. With eACO-VRP, route-aware feasibility, travel cost, service windows, and AP capacity coupling may introduce small non-monotone perturbations. By the bounded approximation-error definition, any additional utility gain caused by such perturbations and pricing approximation is at most ϵ_k .

Critical-reward-like payment. For each winning AP v_k , Step 3 treats its effective ask as a variable and performs binary search to find the largest effective ask at which v_k still remains in the winning set. The resulting value is used to determine the reward $r_{i,k}^{\text{ES}}$ within the settlement band. Under exact monotonicity and exact critical-value pricing, this reward would eliminate profitable unilateral ask deviations. Under

approximate monotonicity and finite-precision binary search, all remaining deviation gains are bounded by ϵ_k . Thus, for any misreported ask $\hat{a}_k$,

$$U_k(a_k; a_k, \mathbf{a}_{-k}) \geq U_k(a_k; \hat{a}_k, \mathbf{a}_{-k}) - \epsilon_k. \quad (39)$$

Combining the ES and AP sides, for every participant q and every unilateral misreport $\hat{\theta}_q$, we have

$$U_q(\theta_q; \theta_q, \mathbf{b}_{-q}) \geq U_q(\theta_q; \hat{\theta}_q, \mathbf{b}_{-q}) - \epsilon_q. \quad (40)$$

Taking the supremum over all possible misreports and using $\epsilon = \max_q \epsilon_q$, we obtain

$$U_q(\theta_q; \theta_q, \mathbf{b}_{-q}) \geq \sup_{\hat{\theta}_q \in \Theta_q} U_q(\theta_q; \hat{\theta}_q, \mathbf{b}_{-q}) - \epsilon. \quad (41)$$

Therefore, Off-AIC² is ϵ -near-truthful for both ESs and APs in the offline stage. This result should be interpreted as an approximate incentive guarantee rather than strict dominant-strategy truthfulness. The numerical evidence in Figs. 2(e)–2(f) further supports this analysis by showing that the realized utility curves are maximized near truthful reports and that unilateral deviations bring at most marginal gains in the tested instances. ■

Proposition 4 (Weak Budget Balance of Off-AIC²). *Under the settlement condition in (27), Off-AIC² satisfies weak budget balance in the offline stage.*

Proof. To verify weak budget balance, it suffices to show that the auctioneer's utility in the offline stage is nonnegative. The auctioneer's utility is

$$U^{\text{Auc, Off}} = \sum_{s_i \in \mathcal{S}} \sum_{v_k \in \mathcal{V}} x_{i,k}^{\text{Off}} N_{i,k}^{\text{Trad}} (p_{i,k}^{\text{ES}} - r_{i,k}^{\text{ES}}). \quad (42)$$

Since $x_{i,k}^{\text{Off}} \in \{0, 1\}$ and $N_{i,k}^{\text{Trad}} \geq 0$, it is enough to prove that every active trading pair satisfies

$$p_{i,k}^{\text{ES}} - r_{i,k}^{\text{ES}} \geq 0. \quad (43)$$

For every active pair (i, k) , the final settlement rule in (27) gives

$$r_{i,k}^{\text{ES}} \leq p_{i,k}^{\text{ES}}. \quad (44)$$

Therefore,

$$x_{i,k}^{\text{Off}} N_{i,k}^{\text{Trad}} (p_{i,k}^{\text{ES}} - r_{i,k}^{\text{ES}}) \geq 0. \quad (45)$$

Unmatched pairs have $x_{i,k}^{\text{Off}} = 0$ and thus contribute zero. Summing over all ES–AP pairs yields

$$U^{\text{Auc, Off}} = \sum_{i,k} x_{i,k}^{\text{Off}} N_{i,k}^{\text{Trad}} (p_{i,k}^{\text{ES}} - r_{i,k}^{\text{ES}}) \geq 0. \quad (46)$$

Therefore, the total payment collected from winning ESs is no smaller than the total reward paid to winning APs, and Off-AIC² satisfies weak budget balance in the offline stage. ■

C. Evaluation and Limitations of eACO-VRP

This appendix evaluates eACO-VRP as an offline or rolling-horizon AP route planner operating at the trading-round timescale. Slot-level SD–SP assignment is handled by PG-BRD. At the beginning of each trading round, Pro-LNN first predicts the ES-side demand over the upcoming service window. Given the predicted demand, AP states, mobility constraints,

service-window constraints, and capacity constraints, eACO-VRP generates feasible AP service routes before online task execution starts. During the subsequent online service window, the generated AP routes and ES–AP contracts are treated as prepared service resources, while fast-timescale variations in HU/MU arrivals, channel states, SD locations, and congestion states are handled by the online PG-BRD scheduler.

This design follows the operational structure of air–ground service provisioning. AP/UAV routes cannot be arbitrarily redesigned at every online slot because vehicle movement, UAV dispatching, service-window feasibility, and UAV recovery operations impose physical constraints. In contrast, SD–SP task assignment can be updated more frequently according to the observed online states. Therefore, eACO-VRP is responsible for route-aware AP preparation at the trading-round timescale, whereas PG-BRD provides fine-grained online adaptability at the slot level. To evaluate whether eACO-VRP is suitable for this intended role, we conduct a rolling-horizon robustness and scalability study. We compare four route-planning methods. *eACO-RH* denotes the proposed rolling-horizon eACO-VRP, which replans AP routes at the beginning of each trading round based on the latest predicted demand. *eACO-Static* denotes a one-shot planner that uses the initial eACO route plan and keeps the same routes in later rounds; later rounds only re-evaluate the fixed routes under current demand and perturbations. *Greedy-RH* and *Random-RH* are lightweight rolling-horizon baselines that construct routes greedily or randomly from feasible ES candidates.

We consider four representative scenarios. The small-scale setting contains 8 ESs, 4 APs, and 80 SDs, including 40 HUs and 40 MUs. The medium-scale setting contains 16 ESs, 8 APs, and 160 SDs, including 80 HUs and 80 MUs. The demand-drift setting contains 12 ESs, 6 APs, and 120 SDs, including 60 HUs and 60 MUs; it applies a drift intensity of 0.3 by blending the original demand vector with a spatially shifted demand vector and adding lognormal temporal perturbations. The topology-perturbation setting also contains 12 ESs, 6 APs, and 120 SDs, but applies a 20% multiplicative perturbation to the route/travel-cost matrix. Each scenario is evaluated over 5 rolling trading rounds and 10 random seeds, yielding 50 seed-round observations for each method. We report four metrics. *Runtime* measures the route-planning wall-clock time per trading round. *Feasible route ratio* measures the fraction of generated routes satisfying route, service-window, and capacity feasibility. *Route utility* is the cumulative AP-side route utility induced by the selected ES visiting sequence. *Normalized SW* is the downstream system social welfare after applying the generated routes, normalized by the eACO-RH result under the same scenario, seed, and trading round.

Table VI shows that eACO-RH consistently achieves the largest route utility and the highest normalized SW across all tested scenarios. Its runtime remains below one second even in the medium-scale case, which is acceptable for trading-round-level offline planning. Compared with eACO-Static, eACO-RH achieves clear gains under both demand drift and topology perturbation, showing that rolling-horizon replanning is necessary when the predicted demand or travel-cost structure changes across trading rounds. Greedy-RH is competitive in

some scenarios due to its very low runtime and simple local selection rule, but eACO-RH remains consistently better in route utility and provides a more systematic search over route-feasible ES combinations. Random-RH samples from feasible ES candidates without prioritizing route utility and achieves lower route utility and normalized SW in the tested scenarios. These results support the use of eACO-VRP as the route-aware AP planning component in FUSION. The offline AP planning problem has a combinatorial VRP-like structure with route feasibility, service-window constraints, heterogeneous AP costs, and capacity limits. eACO-VRP directly targets this structured

route-construction problem and produces feasible AP routes that can be naturally consumed by Off-AIC² for ES-AP contracting. More importantly, once the AP routes and reserved capacities are prepared, PG-BRD can adapt online SD-SP assignment to real-time arrivals and congestion states. Hence, the performance of FUSION comes from the complementary roles of eACO-VRP and PG-BRD: eACO-VRP prepares high-quality route-feasible service resources at the slow timescale, while PG-BRD performs congestion-aware task assignment at the fast timescale.

Nevertheless, eACO-VRP has limitations. If abrupt road blockages, emergency events, severe topology changes, or post-

TABLE VI
ROLLING-HORIZON ROBUSTNESS AND SCALABILITY OF EACO-VRP. NORMALIZED SW IS COMPUTED RELATIVE TO EACO-RH UNDER THE SAME SCENARIO, SEED, AND TRADING ROUND.

Scenario	Method	Runtime	Feasible Route Ratio	Route Utility	Normalized SW
Small scale	eACO-RH	0.0610 $\pm$ 0.0087s	1.000 $\pm$ 0.000	18.26 $\pm$ 2.03	1.000 $\pm$ 0.000
Small scale	eACO-Static	0.0008 $\pm$ 0.0003s	1.000 $\pm$ 0.000	-1.57 $\pm$ 3.16	0.785 $\pm$ 0.034
Small scale	Greedy-RH	0.0001 $\pm$ 0.0000s	1.000 $\pm$ 0.000	18.05 $\pm$ 2.05	0.996 $\pm$ 0.007
Small scale	Random-RH	0.0001 $\pm$ 0.0000s	1.000 $\pm$ 0.000	14.72 $\pm$ 2.19	0.962 $\pm$ 0.020
Medium scale	eACO-RH	0.1901 $\pm$ 0.0164s	1.000 $\pm$ 0.000	36.79 $\pm$ 3.14	1.000 $\pm$ 0.000
Medium scale	eACO-Static	0.0051 $\pm$ 0.0007s	1.000 $\pm$ 0.000	-3.63 $\pm$ 9.37	0.786 $\pm$ 0.057
Medium scale	Greedy-RH	0.0004 $\pm$ 0.0002s	1.000 $\pm$ 0.000	36.12 $\pm$ 3.99	0.996 $\pm$ 0.007
Medium scale	Random-RH	0.0001 $\pm$ 0.0000s	1.000 $\pm$ 0.000	28.69 $\pm$ 3.15	0.956 $\pm$ 0.013
Demand drift	eACO-RH	0.1145 $\pm$ 0.0070s	1.000 $\pm$ 0.000	26.11 $\pm$ 1.60	1.000 $\pm$ 0.000
Demand drift	eACO-Static	0.0023 $\pm$ 0.0004s	1.000 $\pm$ 0.000	-0.01 $\pm$ 5.72	0.810 $\pm$ 0.038
Demand drift	Greedy-RH	0.0002 $\pm$ 0.0001s	1.000 $\pm$ 0.000	26.00 $\pm$ 1.56	0.999 $\pm$ 0.006
Demand drift	Random-RH	0.0001 $\pm$ 0.0000s	1.000 $\pm$ 0.000	21.02 $\pm$ 2.09	0.962 $\pm$ 0.012
Topology perturbation	eACO-RH	0.1115 $\pm$ 0.0094s	1.000 $\pm$ 0.000	26.23 $\pm$ 2.60	1.000 $\pm$ 0.000
Topology perturbation	eACO-Static	0.0023 $\pm$ 0.0005s	1.000 $\pm$ 0.000	-4.51 $\pm$ 5.84	0.775 $\pm$ 0.053
Topology perturbation	Greedy-RH	0.0002 $\pm$ 0.0001s	1.000 $\pm$ 0.000	25.48 $\pm$ 3.16	0.994 $\pm$ 0.011
Topology perturbation	Random-RH	0.0001 $\pm$ 0.0000s	1.000 $\pm$ 0.000	20.99 $\pm$ 2.50	0.959 $\pm$ 0.014

TABLE VII
ROLES OF DIFFERENT COMPONENTS IN THE FUSION FRAMEWORK.

Component	Information state / timescale	Decision role in FUSION	Reason for adoption
Pro-LNN	Before task arrivals; only historical records and aggregate ES-side demand traces are available.	Predicts future ES-side aggregate demand for proactive offline planning.	Purely online scheduling cannot anticipate future overloaded ESs or prepare AP resources in advance.
eACO-VRP	Before the online service window; predicted demand, AP states, mobility/resource constraints, and service-window information are available.	Constructs route-feasible AP service plans and ES visiting sequences.	The AP preparation problem has a combinatorial vehicle routing problem (VRP)-like structure with route feasibility, service-window, and capacity constraints.
Off-AIC ²	Offline contracting stage; ES/AP bids, asks, capacities, predicted demands, and route-feasible AP plans are available.	Forms incentive-aware ES-AP contracts and computes payments/rewards.	Route planning alone does not determine economic incentives, while contracting without route feasibility may produce physically infeasible agreements.
PG-BRD	Online execution stage; actual HU/MU arrivals, SD locations, channel states, and congestion states are observed.	Refines the real-time SD-SP assignment profile π .	Provides distributed congestion-aware online scheduling with convergence under the exact-potential-game structure.

disaster infrastructure failures occur within a service window, the pre-planned AP route may become suboptimal or infeasible. In such cases, event-triggered replanning, dynamic ACO, model predictive control, or multi-agent reinforcement learning could be incorporated to improve real-time responsiveness. Accordingly, eACO-VRP updates AP routes between trading rounds, while PG-BRD adapts SD-SP assignments during online execution.

APPENDIX C RATIONALE FOR THE HYBRID DESIGN OF FUSION

FUSION is modularized according to the information structure and decision dependencies of the considered air-ground edge service provisioning problem. Before the online service window starts, the system only has access to historical demand records, predicted ES-side aggregate demand, AP states, mobility/resource constraints, and ES/AP-side bidding information. Therefore, decisions that must be prepared in advance, including AP route planning, service-capacity reservation, and ES-AP contracting, are handled in the offline stage. After actual HU/MU arrivals, SD locations, channel states, and congestion states are revealed, the online stage performs real-time SD-SP assignment through PG-BRD.

The four components are connected through their decision dependencies. Pro-LNN supplies demand forecasts, eACO-VRP constructs route-feasible service plans, Off-AIC² determines ES-AP contracts and settlement terms, and PG-BRD updates SD-SP assignments after online states become available. Route feasibility and economic admissibility constrain the offline service commitments, whereas realized task and congestion states determine the online assignments. Table VII summarizes these roles.

APPENDIX D PROPERTIES OF THE EXACT POTENTIAL

This appendix states the exact-potential property of PG-BRD for the potential-aligned payoff in (19). Under this payoff, every unilateral feasible deviation produces the same change in the SD's payoff and in the game potential $\Phi^G(\pi)$ defined in (21). The result concerns the potential-aligned scheduling payoff, not the raw HU/MU utility alone.

Proposition 5 (Exact potential and finite improvement). *Consider the online SD-side scheduling game in which each SD u_j^{On} chooses one feasible action from $\mathcal{A}_j^{\text{feas}}$ and evaluates each action according to the potential-aligned payoff in (19). The posted prices $\{p_{j,i}\}$, WPS weights $\{\gamma_{j,i}\}$, coverage sets, and capacity limits are fixed throughout one PG-BRD run. Then $\Phi^G(\pi)$ in (21) is an exact potential. Specifically, for any SD u_j^{On} and any unilateral deviation from action i to action i' ,*

$$\tilde{U}_{j,i'}(\pi_{-j}) - \tilde{U}_{j,i}(\pi_{-j}) = \Phi^G(\pi_{-j}, i') - \Phi^G(\pi_{-j}, i). \quad (47)$$

Therefore, every accepted payoff-improving update strictly increases $\Phi^G(\pi)$. Since the feasible joint action space is finite, PG-BRD with $\varepsilon = 0$ terminates at a pure-strategy NE. With a positive threshold $\varepsilon > 0$, termination without accepted updates yields an ε -NE. If $T_{\max}$ is reached before the convergence

condition is satisfied, PG-BRD returns the final feasible profile without an equilibrium guarantee.

The full proof is given in Appx. E. In particular, Appx. E.3 proves the exact-potential identity, Appx. E.4 explains the active-regime MU soft-deadline form, Appx. E.5 proves the finite-improvement and ε -NE claims, and Appx. E.6 clarifies the distinction between transfer-free welfare and the game potential.

APPENDIX E PROOF DETAILS FOR THE EXACT POTENTIAL AND PG-BRD

This appendix proves the exact identity between unilateral changes in the potential-aligned scheduling payoff and changes in the game potential, followed by the finite-improvement and ε -NE results.

A. Matrix Formulation Based on the Global Assignment Matrix

For implementation and analysis, we represent the joint SD assignment profile by a global one-hot matrix

$$\mathbf{X} \in \{0, 1\}^{|\mathcal{U}^{\text{On}}| \times (1 + |\mathcal{S}^{\text{On}}|)}. \quad (48)$$

The j -th row is

$$\mathbf{X}_j^{\text{On}} = (x_{j,0}^{\text{On}}, x_{j,1}^{\text{On}}, \dots, x_{j,|\mathcal{S}^{\text{On}}|}^{\text{On}}), \quad (49)$$

where $x_{j,0}^{\text{On}} = 1$ denotes local execution and $x_{j,i}^{\text{On}} = 1$ denotes offloading to SP s_i^{On} . The one-hot constraint is

$$\sum_{i=0}^{|\mathcal{S}^{\text{On}}|} x_{j,i}^{\text{On}} = 1, \quad \forall u_j^{\text{On}} \in \mathcal{U}^{\text{On}}. \quad (50)$$

Equivalently, the scalar action π_j and the one-hot vector $\mathbf{X}_j^{\text{On}}$ represent the same decision.

For each online SP s_i^{On} , the effective congestion load and physical workload induced by $\mathbf{X}$ are

$$y_i(\mathbf{X}) = \sum_{j \in \mathcal{U}^{\text{On}}} \gamma_{j,i} x_{j,i}^{\text{On}}, \quad \mathbb{x}_i(\mathbf{X}) = \sum_{j \in \mathcal{U}^{\text{On}}} r_j x_{j,i}^{\text{On}}. \quad (51)$$

These quantities coincide with $y_i(\pi)$ and $\mathbb{x}_i(\pi)$ in (20) whenever the rows of $\mathbf{X}$ are induced by the scalar profile π .

B. Marginal-Contribution Form of the Scheduling Payoff

Fix an SD u_j^{On} and the action profile of all other SDs, denoted by π_{-j} . For each offloading action $i \in \mathcal{S}^{\text{On}}$, define the load and workload generated by all SDs except u_j^{On} as

$$y_i^{-j} = \sum_{v \neq j: \pi_v = i} \gamma_{v,i}, \quad \mathbb{x}_i^{-j} = \sum_{v \neq j: \pi_v = i} r_v. \quad (52)$$

The marginal congestion contribution of assigning u_j^{On} to s_i^{On} is

$$C_{j,i}(\pi_{-j}) = \int_{y_i^{-j}}^{y_i^{-j} + \gamma_{j,i}} \tau_i(z) dz, \quad (53)$$

and the marginal SP-side operating cost is

$$\mathcal{K}_{j,i}(\pi_{-j}) = K_i(\mathbb{x}_i^{-j} + r_j) - K_i(\mathbb{x}_i^{-j}). \quad (54)$$

For local execution, we use the convention

$$S_j(0) = 0, \quad p_{j,0} = 0, \quad C_{j,0}(\pi_{-j}) = 0, \quad \mathcal{K}_{j,0}(\pi_{-j}) = 0. \quad (55)$$

Then the potential-aligned payoff in (19) can be written compactly as

$$\tilde{U}_{j,i}(\pi_{-j}) = S_j(i) - C_{j,i}(\pi_{-j}) - \mathcal{K}_{j,i}(\pi_{-j}) - p_{j,i}, \quad (56)$$

for every action $i \in \{0\} \cup \mathcal{S}^{\text{On}}$.

C. Single-User Deviation and Exact Potential Identity

We now prove the exact-potential identity. Fix π_{-j} and consider a unilateral deviation of SD u_j^{On} from action i to action i' . Denote the two resulting profiles by

$$\pi = (\pi_{-j}, i), \quad \pi' = (\pi_{-j}, i'). \quad (57)$$

All terms in Φ^G that are unrelated to actions i and i' remain unchanged. Therefore, the change in the valuation term is

$$\Delta\Phi_{\text{val}} = S_j(i') - S_j(i). \quad (58)$$

The congestion-integral change is

$$\Delta\Phi_{\text{cong}} = -C_{j,i'}(\pi_{-j}) + C_{j,i}(\pi_{-j}). \quad (59)$$

Indeed, assigning u_j^{On} to the new offloading action i' increases the integrated congestion at $s_{i'}^{\text{On}}$ by $C_{j,i'}$, while removing it from the old offloading action i decreases the integrated congestion at s_i^{On} by $C_{j,i}$. The local-execution case is covered by the zero convention above.

Similarly, the SP-cost change is

$$\Delta\Phi_{\text{cost}} = -\mathcal{K}_{j,i'}(\pi_{-j}) + \mathcal{K}_{j,i}(\pi_{-j}). \quad (60)$$

This expression is exact for any cost function $K_i(\cdot)$ with finite values; it does not require differentiability or convexity. Hence, it remains valid when $K_i(\cdot)$ includes the activation cost $\omega_2 c_i^{\text{hard}} \mathbb{1}_{\{x_i > 0\}}$ in (16).

Finally, the posted-price term changes by

$$\Delta\Phi_{\text{price}} = -p_{j,i'} + p_{j,i}. \quad (61)$$

Adding the four components gives

$$\begin{aligned} \Phi^G(\pi') - \Phi^G(\pi) &= S_j(i') - C_{j,i'}(\pi_{-j}) - \mathcal{K}_{j,i'}(\pi_{-j}) - p_{j,i'} \\ &\quad - \left[S_j(i) - C_{j,i}(\pi_{-j}) - \mathcal{K}_{j,i}(\pi_{-j}) - p_{j,i} \right]. \end{aligned} \quad (62)$$

Using (56), we obtain

$$\Phi^G(\pi_{-j}, i') - \Phi^G(\pi_{-j}, i) = \tilde{U}_{j,i'}(\pi_{-j}) - \tilde{U}_{j,i}(\pi_{-j}). \quad (63)$$

This proves that Φ^G is an exact potential function for the PG-BRD payoff.

D. Active-Regime Form of the MU Soft-Deadline Utility

We next clarify how the MU soft-deadline utility connects to the non-congestion valuation term $S_j(i)$. For MU u_j^{On} offloading to s_i^{On} ,

$$G_{j,i}^M = v_j \frac{[\tau_{j,i}^{\text{ddl}} - T_{j,i}^{\text{edge}}]_+}{\tau_{j,i}^{\text{ddl}}}, \quad T_{j,i}^{\text{edge}} = t_{j,i}^{\text{tx}} + \theta_{j,i} \tau_i(y_i). \quad (64)$$

In the active soft-deadline regime $T_{j,i}^{\text{edge}} < \tau_{j,i}^{\text{ddl}}$, the positive-part operator is active, and thus

$$G_{j,i}^M = v_j - v_j \frac{t_{j,i}^{\text{tx}}}{\tau_{j,i}^{\text{ddl}}} - \frac{v_j}{\tau_{j,i}^{\text{ddl}}} \theta_{j,i} \tau_i(y_i). \quad (65)$$

After adding the MU energy-saving term $\mathbb{V}_e^M c_{j,i}^{\text{M,save}}$ and subtracting the posted price, the active-regime MU utility becomes

$$U_{j,i}^M = \bar{S}_j(i) - \kappa_j \theta_{j,i} \tau_i(y_i) - p_{j,i}, \quad (66)$$

where

$$\bar{S}_j(i) = v_j - v_j \frac{t_{j,i}^{\text{tx}}}{\tau_{j,i}^{\text{ddl}}} + \mathbb{V}_e^M c_{j,i}^{\text{M,save}}, \quad \kappa_j = \frac{v_j}{\tau_{j,i}^{\text{ddl}}}. \quad (67)$$

Therefore, in the active regime, MUs share the same congestion-sensitive structure as HUs. Outside this regime, the slope of the positive-part operator becomes zero. In implementation, the original piecewise soft-deadline expression in (14) can be evaluated directly, while the active-regime form explains the congestion sensitivity used to construct the potential-aligned payoff.

E. Finite-Improvement Property and ε -NE Guarantee

The exact-potential identity in Appx. E.3 implies the finite-improvement property. Each SD has a finite feasible action set because it can only choose local execution or one accessible SP satisfying the coverage and capacity constraints. Hence, the joint feasible action space is finite.

When $\varepsilon = 0$, PG-BRD accepts only strict payoff-improving unilateral deviations. By the exact-potential identity, each accepted update strictly increases $\Phi^G(\pi)$. Since the feasible profile space is finite, the algorithm cannot generate an infinite sequence of distinct profiles with strictly increasing potential values. It must terminate at a profile π^* where no SD has a strictly profitable unilateral feasible deviation. By definition, π^* is a pure-strategy NE of the potential-aligned scheduling game.

When PG-BRD uses a positive threshold $\varepsilon > 0$, termination with converged = **true** means that for every SD u_j^{On} and every feasible action $a \in \mathcal{A}_j^{\text{feas}}$,

$$\tilde{U}_j(a | \pi_j^*) - \tilde{U}_j(\pi_j^* | \pi_j^*) \leq \varepsilon. \quad (68)$$

Thus, no SD can improve its potential-aligned payoff by more than ε through a unilateral feasible action change, and π^* is an ε -NE. If the algorithm stops because T_{max} is reached before converged = **true**, no equilibrium guarantee is claimed.

F. Transfer-Free Welfare and Game Potential

The transfer-free welfare metric and the game potential play different roles. The online welfare metric aggregates SD-side non-congestion valuations and subtracts integrated congestion costs and SP-side operating costs:

$$\mathcal{W}(\pi) = \sum_{j: \pi_j \neq 0} S_j(\pi_j) - \sum_{i \in \mathcal{S}^{\text{On}}} \int_0^{y_i(\pi)} \tau_i(z) dz - \sum_{i \in \mathcal{S}^{\text{On}}} K_i(x_i(\pi)). \quad (69)$$

This metric excludes monetary transfers between SDs and SPs and is used for online welfare evaluation.

TABLE VIII
COMPARISON WITH DRL-BASED ONLINE SCHEDULERS UNDER THE SAME OFFLINE PLANNING RESULTS.

Method	Social Welfare	Timely Ratio	Served Ratio	Avg. Served Delay	Energy Cost	Online Runtime	Training Time
FUSION	9362.50 $\pm$ 552.09	0.769 $\pm$ 0.048	0.876 $\pm$ 0.035	2.929 $\pm$ 0.084	1470.73 $\pm$ 308.62	0.0909 $\pm$ 0.0415s	None
FUSION-DQN	8946.71 $\pm$ 710.14	0.711 $\pm$ 0.056	0.921 $\pm$ 0.047	3.214 $\pm$ 0.113	1057.22 $\pm$ 248.76	0.0307 $\pm$ 0.0046s	7.32 $\pm$ 0.46s (100 ep)
FUSION-PPO	8848.63 $\pm$ 731.95	0.700 $\pm$ 0.058	0.919 $\pm$ 0.045	3.265 $\pm$ 0.093	1068.20 $\pm$ 263.83	0.0397 $\pm$ 0.0043s	27.54 $\pm$ 2.46s (100 ep)
FUSION-MADDPG	8904.83 $\pm$ 727.07	0.697 $\pm$ 0.054	0.921 $\pm$ 0.047	3.313 $\pm$ 0.084	1051.14 $\pm$ 260.71	0.0474 $\pm$ 0.0156s	52.52 $\pm$ 5.06s (100 ep)
FUSION_NoPG	7781.43 $\pm$ 749.96	0.642 $\pm$ 0.052	0.843 $\pm$ 0.056	3.310 $\pm$ 0.131	1517.06 $\pm$ 330.30	0.0040 $\pm$ 0.0003s	None
PurOnline	6565.13 $\pm$ 545.01	0.525 $\pm$ 0.018	0.625 $\pm$ 0.022	3.222 $\pm$ 0.192	3017.13 $\pm$ 227.11	0.0281 $\pm$ 0.0030s	None

In contrast, the game potential $\Phi^G(\pi)$ includes posted prices because posted prices affect SD-side best responses. If prices were omitted from the game potential while SDs optimized price-dependent utilities, a price-driven unilateral improvement could fail to increase the potential. Including the fixed posted price term in (21) preserves the exact-potential identity while keeping $\mathcal{W}(\pi)$ available as the transfer-free welfare metric.

APPENDIX F PERFORMANCE EVALUATION ON DRL-BASED ONLINE SCHEDULERS

This appendix provides an additional comparison between FUSION and representative DRL-based online scheduling baselines. The goal is to examine whether learning-based online policies can replace the PG-BRD scheduler when the same offline service preparation is available. For a fair comparison, all DRL baselines replace only the online PG-BRD component, while using the same offline outputs as FUSION, including the Pro-LNN demand forecasts, eACO-VRP AP routes, Off-AIC² ES-AP contracts, and reserved service capacities. Thus, FUSION and its DRL variants operate under the same predicted demand, AP/ES resource configuration, mobility setting, channel model, online task-arrival process, and feasibility constraints. The performance difference therefore reflects the impact of the online scheduling policy rather than differences in offline planning.

We consider three representative DRL baselines. *DQN* is a value-based method for discrete online assignment decisions. *PPO* is a policy-gradient-based method with clipped policy updates and improved training stability. *MADDPG* is a representative multi-agent DRL method for distributed decision-making with centralized training. For all DRL baselines, the state includes online task-arrival information, accessible SP sets, residual subcarrier capacity, estimated delay, energy-related features, and congestion/load indicators. The action corresponds to selecting local execution or one feasible SP. The reward is aligned with the online system objective by combining service utility, deadline/timeliness reward, delay penalty, energy cost, and capacity-violation penalty.

Table VIII shows that FUSION achieves the highest social welfare and timely ratio among all compared methods. Compared with FUSION-DQN, FUSION improves social welfare from 8946.71 to 9362.50 and timely ratio from 0.711 to 0.769. Similar gains are observed over FUSION-PPO and FUSION-MADDPG. These results indicate that PG-BRD better balances service value, delay sensitivity, deadline satisfaction, congestion effects, and feasibility constraints under the same offline service preparation. The DRL baselines achieve higher served ratios

and lower energy costs than FUSION. This suggests that learned policies tend to admit or serve more tasks with lower energy expenditure. However, their average served delay is larger and their timely ratios are lower, indicating that they do not always prioritize delay-sensitive or deadline-critical tasks in a way that maximizes the overall system utility. In contrast, PG-BRD explicitly exploits the exact-potential structure of the online scheduling game and updates SD-SP assignments according to potential-aligned payoff-improving best responses, which leads to higher timely service performance and social welfare. Another important difference lies in training and interpretability. DQN, PPO, and MADDPG require additional policy training before deployment, and their performance depends on the coverage and quality of training samples. When the task-arrival distribution, mobility pattern, channel condition, or AP/ES resource configuration changes, the learned policies may need to be retrained or fine-tuned. By contrast, PG-BRD does not require offline policy training. It directly operates on the currently observed online state and converges under the finite-potential-game structure. Although PG-BRD has a slightly higher online runtime than the trained DRL policies in this experiment, its runtime remains below 0.1 seconds on average and avoids the additional training cost.

Under identical offline service preparation, PG-BRD achieves higher SW and timely ratios without online-policy training, whereas the tested DRL policies achieve higher served ratios, lower energy costs, and shorter online runtimes. All variants retain the same offline contracting component, so this comparison isolates the online scheduling tradeoff rather than differences in offline economic properties.

APPENDIX G DECOMPOSITION-INDUCED OPTIMALITY GAP AND RUNTIME

This appendix compares FUSION with small-scale full-information benchmarks to evaluate the performance-runtime tradeoff of the two-stage design. FUSION prepares forecasts, AP routes, and ES-AP contracts before online task arrivals and updates SD-SP assignments after online states become available.

To evaluate the decomposition-induced gap, we construct small-scale oracle benchmark instances in which future task arrivals, channel states, route feasibility, and system states are assumed to be known. For each instance, an exact branch-and-bound solver is first attempted to solve the corresponding full-information benchmark. If the exact solver terminates within the time limit, the obtained optimum is used as the benchmark value. Otherwise, we use the relaxed upper bound returned

TABLE IX
PERFORMANCE GAP AND RUNTIME COMPARISON BETWEEN FUSION AND SMALL-SCALE FULL-INFORMATION BENCHMARKS.

Case	Problem Size	Benchmark Type	Exact/UB SW	FUSION SW	Gap	Exact/UB Runtime	FUSION Runtime
1	$ \mathcal{U} = 10, \mathcal{S} = 3, \mathcal{V} = 2$	7/10 Exact, 3/10 UB	982.50 ± 172.22	974.64 ± 173.01	$0.83 \pm 1.13\%$	$0.9005 \pm 0.9207s$	$0.0016 \pm 0.0006s$
2	$ \mathcal{U} = 16, \mathcal{S} = 4, \mathcal{V} = 2$	2/10 Exact, 8/10 UB	1615.33 ± 223.16	1580.05 ± 224.43	$2.25 \pm 1.73\%$	$1.7800 \pm 0.5200s$	$0.0028 \pm 0.0006s$
3	$ \mathcal{U} = 20, \mathcal{S} = 5, \mathcal{V} = 3$	0/10 Exact, 10/10 UB	2090.58 ± 266.99	2041.00 ± 272.14	$2.44 \pm 0.74\%$	$2.0003 \pm 0.0001s$	$0.0050 \pm 0.0017s$

by the solver as the benchmark value. Therefore, when the benchmark type contains upper-bound cases, the reported gap should be interpreted as a conservative upper estimate of the true optimality gap. The gap is computed as

$$\text{Gap} = \frac{U_{\text{SW}}^{\text{Exact/UB}} - U_{\text{SW}}^{\text{FUSION}}}{U_{\text{SW}}^{\text{Exact/UB}}} \times 100\%. \quad (70)$$

Table IX shows that FUSION achieves a small exact/upper-bound gap, ranging from approximately 0.83% to 2.44% across the tested small-scale cases. At the same time, FUSION requires substantially lower runtime than the exact/upper-bound benchmark. For example, in Case 1, FUSION obtains an SW of 974.64 compared with the exact/upper-bound value of 982.50, while reducing the runtime from 0.9005s to 0.0016s. As the problem size increases, exact branch-and-bound becomes increasingly difficult: the benchmark changes from 7/10 exact solutions in Case 1 to 0/10 exact solutions in Case 3, where all benchmark values rely on relaxed upper bounds. This trend illustrates the rapid growth of the full-information joint problem complexity. The tested small-scale instances show limited gaps relative to exact optima or solver upper bounds and substantially lower runtime for FUSION. These comparisons characterize the tradeoff under the specified benchmark settings; they do not establish global optimality or a general optimality-gap bound for larger instances.